BetterGym:
Aplicação Android para descoberta de ginásios

Bruno Saraiva (1250795), Tomás Pereira (1251521)

Instituto Superior de Engenharia do Porto (ISEP)
Licenciatura em Engenharia de Telecomunicações e Informática
Desenvolvimento de Software e Sistemas Móveis – 2026/27 – Turma 2DB – Equipa 8
{1250795, 1251521}@isep.ipp.pt

Resumo: O BetterGym é uma aplicação Android que permite descobrir ginásios próximos através da localização GPS e consultar informações como morada, fotografias, estado de abertura e avaliações públicas. A aplicação disponibiliza autenticação de utilizadores, gestão de favoritos, registo de visitas através da leitura de códigos QR e consulta do histórico de visitas. Desenvolvida em Java, integra os serviços Firebase, Google Maps e Google Places API, combinando recursos do dispositivo móvel com serviços externos para facilitar a procura de ginásios e o acompanhamento da sua frequência.

Palavras-chave: Android, Java, Firebase, Ginásios, Google Maps, Google Places API, QR code, Localização.

1. Introdução

A escolha de um ginásio envolve proximidade, morada, classificação pública e informação de abertura. A consulta destes dados em fontes separadas torna menos direta a comparação de estabelecimentos. Neste âmbito, foi desenvolvida a BetterGym, uma aplicação Android que permite descobrir ginásios através da localização GPS do dispositivo e consultar informações. A aplicação permite ainda guardar ginásios nos favoritos, registar visitas através da leitura de códigos QR e consultar o respetivo histórico, disponibilizando uma área de perfil com dados pessoais e estatísticas de utilização.

O projeto foi realizado no âmbito da unidade curricular de Desenvolvimento de Software e Sistemas Móveis (DSSMV), com o objetivo de aplicar os conhecimentos adquiridos no desenvolvimento de aplicações móveis. A implementação foi efetuada em Java, recorrendo ao Firebase para autenticação e armazenamento de dados e aos serviços Google Maps e Google Places API para apresentação do mapa e obtenção de

informações sobre os ginásios. A integração do GPS e da câmara permite explorar recursos do dispositivo, enquanto o processamento assíncrono das operações de rede contribui para manter a interface responsiva.

O presente relatório descreve o contexto e os objetivos do projeto, a análise de requisitos, as decisões de conceção, a implementação das funcionalidades e a respetiva validação. Por fim, apresenta as conclusões do trabalho desenvolvido, as limitações identificadas e as possibilidades de evolução da aplicação.

2. Análise

2.1. Modelo de Domínio

O modelo de domínio da BetterGym representa os principais conceitos da aplicação e as relações entre eles: Utilizador, Ginásio, Favorito, Visita e Avaliação.

O Utilizador reúne os dados pessoais e a fotografia de perfil, podendo guardar ginásios nos favoritos, registar visitas e consultar avaliações. O Ginásio contém o nome, a morada, as coordenadas geográficas e a classificação pública. O Favorito associa um utilizador a um ginásio e regista a data de adição. A Visita identifica uma ocorrência de frequência de um ginásio, guardando a respetiva data. A Avaliação representa uma avaliação pública disponibilizada pelo Google, incluindo o autor, a classificação e o comentário.

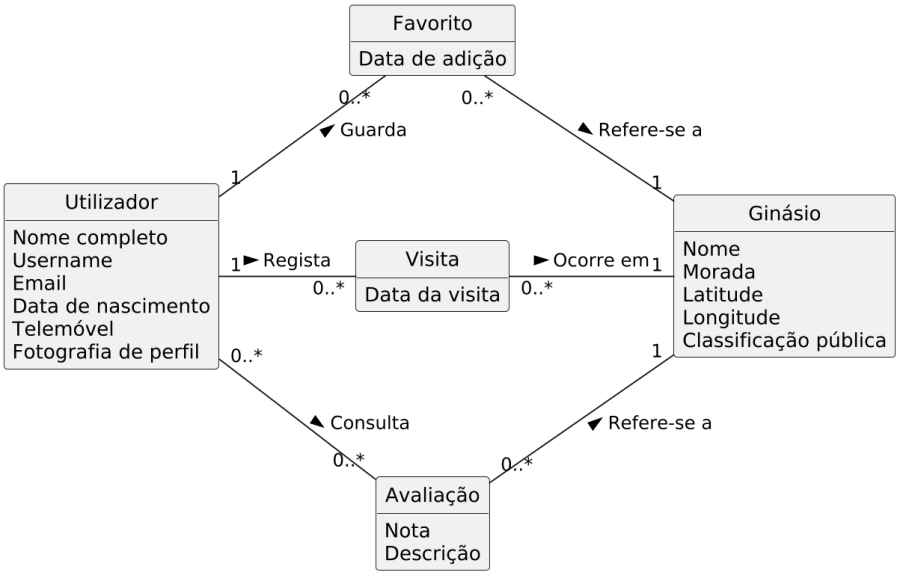


Figura 1 - Modelo de Domínio.

2.2. Requisitos não funcionais

- **Usabilidade:**
  - Interface gráfica intuitiva para Android.
- **Fiabilidade:**
  - Apresentar mensagens de erro perante falhas de autenticação, localização ou comunicação.
- **Desempenho:**
  - Executar operações de rede em segundo plano, sem bloquear a interface.
- **Suportabilidade:**
  - Organização do código em packages;
  - Testes unitários para validar restrições específicas;
  - Plataforma Android;
  - Versão mínima: Android API 25.
- **Restrições de design:**
  - Não foram definidas restrições específicas.
- **Restrições de implementação:**
  - Linguagem Java.
- **Restrições de interface:**
  - Interface gráfica Android definida através de layouts XML.

2.3. Requisitos funcionais

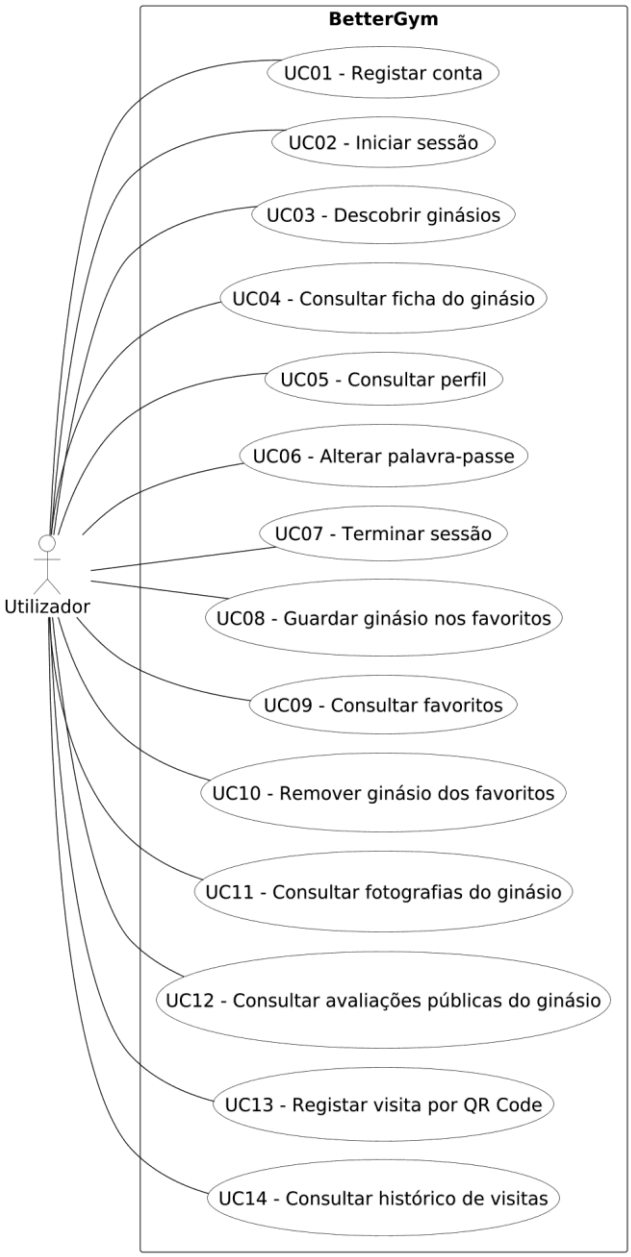


Figura 2 – Casos de uso

2.3.1. UC01 – Registar conta

Descrição	Criar uma conta de utilizador e guardar o respetivo perfil
Pré-Condição	O formulário de registo encontra-se disponível
Pós-Condição	A conta e o perfil ficam criados e é apresentado o ecrã de login
Caminho Básico	**1.** O utilizador introduz os dados pessoais, o email, a palavra-passe e a confirmação; **2.** Opcionalmente, o utilizador escolhe uma foto de perfil; **3.** O utilizador seleciona "Criar conta"; **4.** O sistema valida os dados e cria a conta; **5.** O sistema guarda o perfil; **6.** O sistema confirma o sucesso e apresenta o login.
Caminhos alternativos	**Dados inválidos:** o sistema apresenta uma mensagem de erro e permite corrigir os campos. **Email já utilizado:** o sistema informa que a conta já existe. **Falha no serviço:** o sistema apresenta erro. **Conta criada, mas perfil não guardado:** o sistema apresenta um erro de consistência; a conta de autenticação permanece criada.

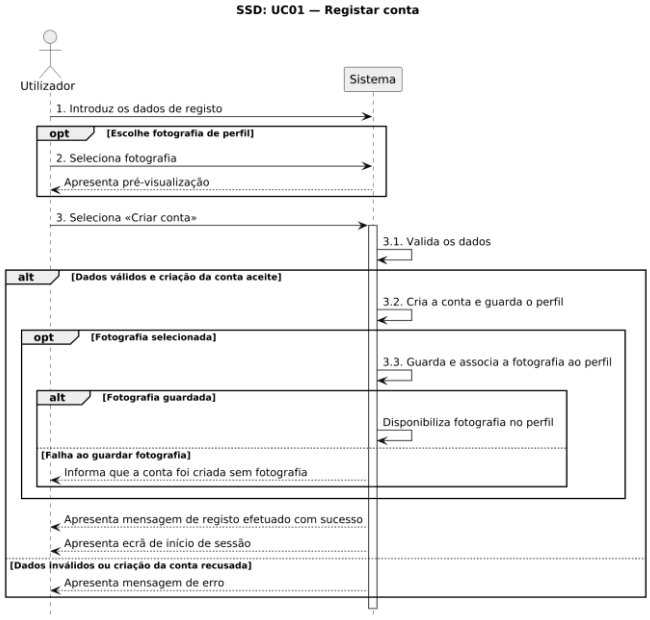


Figura 3 – SSD da UC01.

2.3.2. UC02 – Iniciar sessão

Descrição	Autenticar o utilizador para aceder à aplicação
Pré-Condição	O utilizador possui uma conta e encontra-se no ecrã de login
Pós-Condição	A sessão fica iniciada e é aberto o ecrã principal
Caminho Básico	**1.** O utilizador introduz o email e a palavra-passe; **2.** O utilizador seleciona "Entrar"; **3.** O sistema valida os campos e autentica as credenciais; **4.** O sistema guarda localmente o identificador do utilizador; **5.** O sistema apresenta uma mensagem de sucesso e inicia o carregamento do ecrã principal.
Caminhos alternativos	**Campos vazios:** o sistema solicita o preenchimento dos dados. **Autenticação recusada:** o sistema apresenta erro e permite uma nova tentativa.

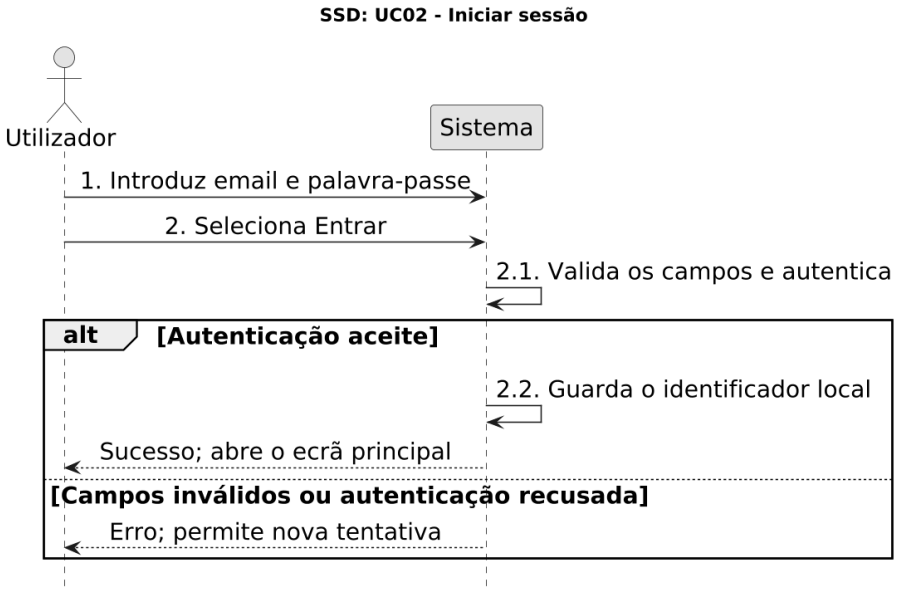


Figura 4 – SSD do UC02.

2.3.3. UC03 – Descobrir ginásios

Descrição	Apresentar no mapa os ginásios próximos da localização do dispositivo
Pré-Condição	O ecrã principal da aplicação encontra-se aberto
Pós-Condição	O mapa apresenta os ginásios encontrados nas proximidades.
Caminho Básico	**1.** O utilizador acede ao mapa; **2.** O sistema solicita permissões de localização, quando necessário; **3.** O utilizador concede a permissão; **4.** O sistema obtém a localização e pesquisa até 20 ginásios num raio de 10 km; **5.** O sistema apresenta os marcadores e conclui o carregamento.
Caminhos alternativos	**Permissão recusada ou localização indisponível:** o sistema apresenta uma mensagem e as opções de repetir ou continuar. **Nenhum ginásio encontrado:** o sistema apresenta o mapa sem marcadores.

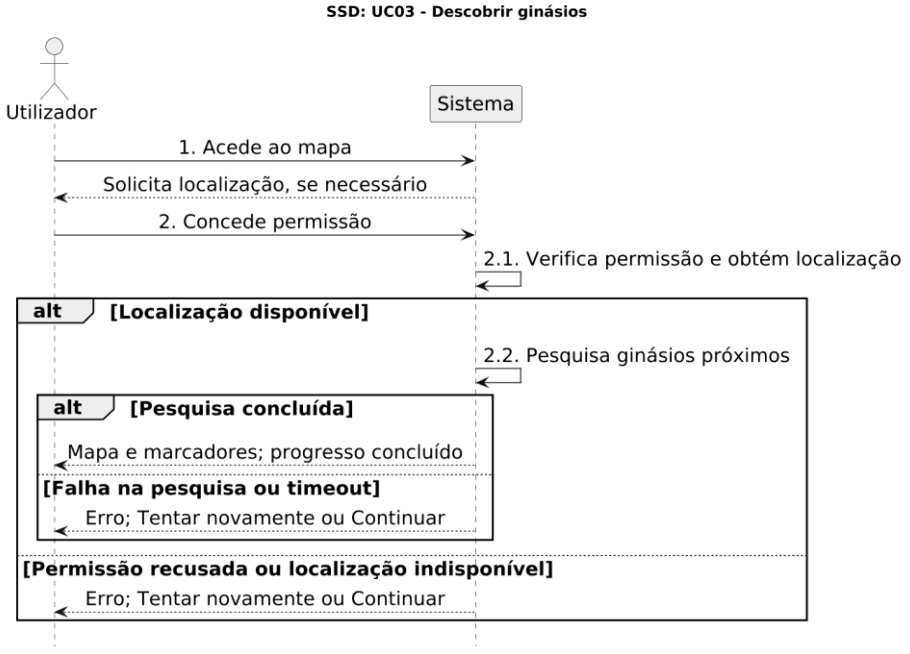


Figura 5 – SSD do UC03

2.3.4. UC04 – Consultar ficha do ginásio

Descrição	Consultar as informações atualizadas de um ginásio
Pré-Condição	Existe um ginásio selecionável no mapa, na lista de favoritos ou no histórico de visitas
Pós-Condição	A ficha de ginásio é apresentada
Caminho Básico	**1.** O utilizador seleciona um ginásio; no mapa, seleciona também "Ver detalhes"; **2.** O sistema verifica o identificador do ginásio; **3.** O sistema obtém os detalhes atualizados; **4.** O sistema apresenta nome, morada, estado de abertura, classificação e número de avaliações; **5.** O sistema disponibiliza as fotografias, avaliações e ações associadas ao ginásio.
Caminhos alternativos	**Identificador em falta:** o sistema apresenta erro e fecha a ficha. **Falha na consulta:** o sistema apresenta erro e regressa ao ecrã anterior. **Fotografias ou avaliações inexistentes:** a ficha apresenta os restantes dados disponíveis.

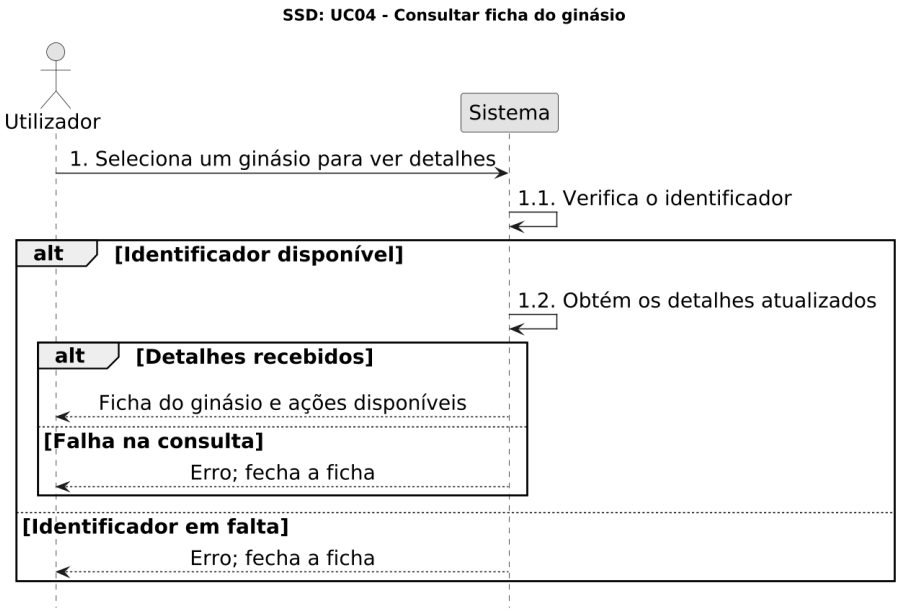


Figura 6 – SSD do UC04

2.3.5. UC05 – Consultar perfil

Descrição	Consultar os dados pessoais e as estatísticas do utilizador
Pré-Condição	O utilizador tem sessão iniciada
Pós-Condição	Os dados do perfil são apresentados sem serem alterados
Caminho Básico	**1.** O utilizador seleciona "Perfil"; **2.** O sistema verifica a sessão; **3.** O sistema obtém os dados do perfil; **4.** O sistema obtém as contagens disponíveis; **5.** O sistema apresenta os dados pessoais e as estatísticas.
Caminhos alternativos	**Sessão indisponível:** o sistema apresenta o login. **Falha ao carregar o perfil:** o sistema apresenta uma mensagem de erro. **Falha na consulta das contagens:** o sistema apresenta 0 nos contadores afetados.

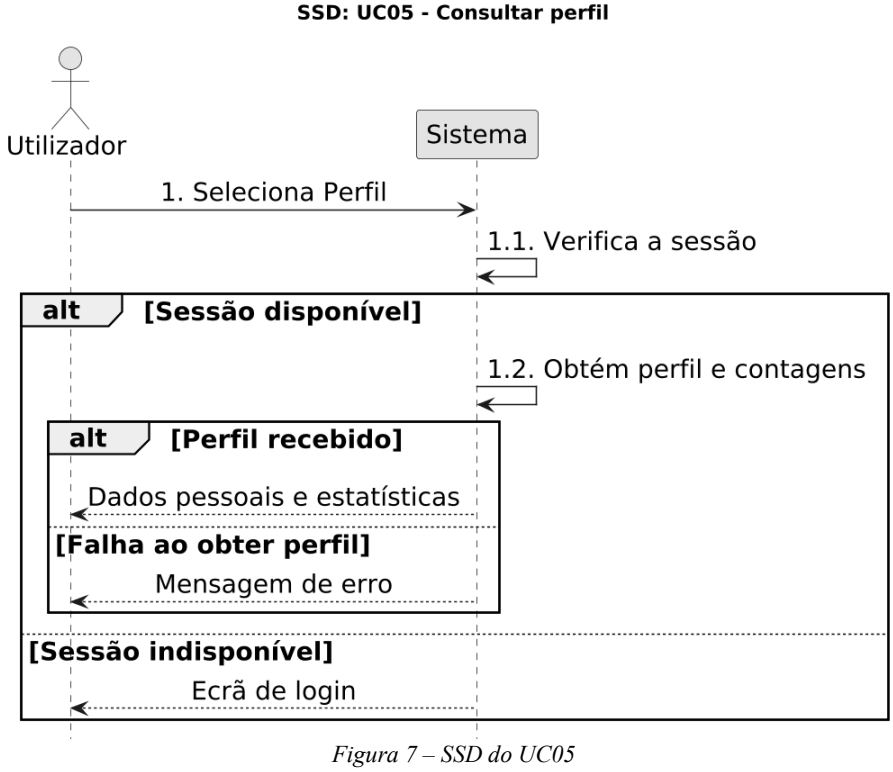


Figura 7 – SSD do UC05

2.3.6. UC06 – Alterar palavra-passe

Descrição	Alterar a palavra-passe mediante a confirmação da palavra-passe atual
Pré-Condição	O utilizador tem sessão iniciada e encontra-se no perfil
Pós-Condição	A nova palavra-passe fica definida e o pop-up é fechado
Caminho Básico	**1.** O utilizador seleciona "Alterar palavra-passe"; **2.** O sistema apresenta o pop-up; **3.** O utilizador introduz a palavra-passe atual, a nova e a confirmação e seleciona "Guardar"; **4.** O sistema valida os campos e confirma a credencial atual; **5.** O sistema atualiza a palavra-passe; 6. O sistema apresenta sucesso, limpa os campos e fecha o pop-up.
Caminhos alternativos	**Cancelar:** o pop-up é fechado sem alterações. **Dados inválidos ou confirmação diferente:** o sistema apresenta erro. **Palavra-passe atual incorreta:** o sistema recusa a alteração. **Falha na atualização:** o sistema apresenta erro e permite repetir.

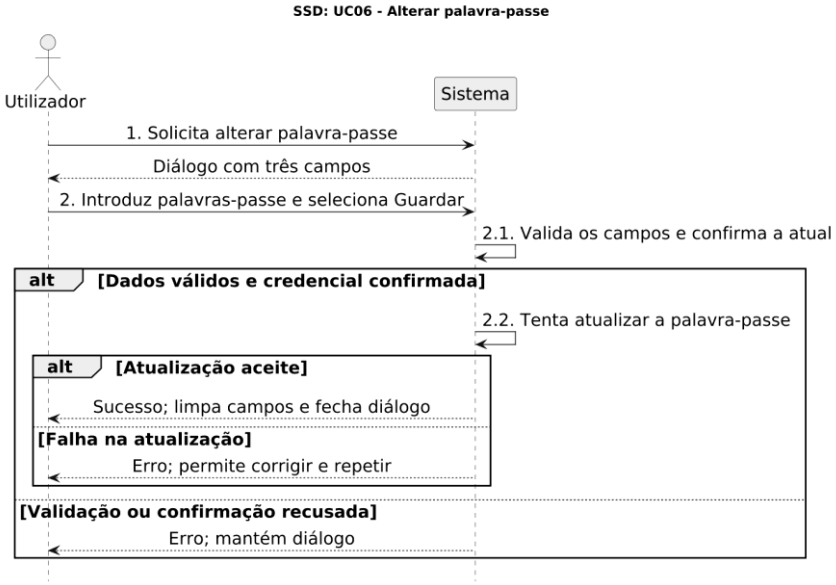


Figura 8 – SSD do UC06

2.3.7. UC07 – Terminar sessão

Descrição	Encerrar a sessão do utilizador
Pré-Condição	O utilizador tem sessão iniciada e encontra-se no perfil
Pós-Condição	A sessão fica terminada e é apresentado o ecrã de login
Caminho Básico	**1.** O utilizador seleciona "Terminar sessão"; **2.** O sistema encerra sessão; **3.** O sistema limpa as preferências locais da aplicação; **4.** O sistema limpa a pilha de navegação e apresenta o login.
Caminhos alternativos	Não existem fluxos alternativos explícitos implementados para esta operação

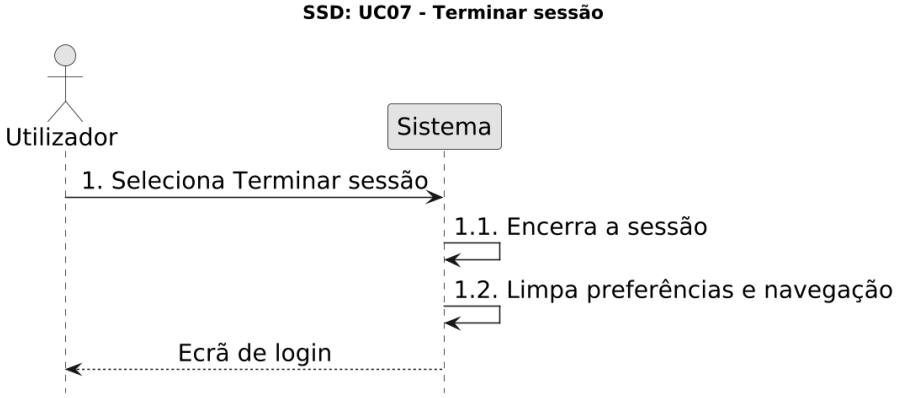


Figura 9 – SSD do UC07

2.3.8. UC08 – Guardar ginásio nos favoritos

Descrição	Adicionar um ginásio aos favoritos do utilizador
Pré-Condição	O utilizador tem sessão iniciada, a ficha está carregada e o ginásio não está guardado nos favoritos
Pós-Condição	O favorito fica guardado com a data de adição e a estrela é apresentada preenchida
Caminho Básico	**1.** O utilizador seleciona a estrela vazia; **2.** O sistema verifica a sessão e os dados do ginásio; **3.** O sistema guarda o favorito associado ao utilizador; **4.** O sistema apresenta uma confirmação e preenche a estrela.
Caminhos alternativos	**Estado do favorito desconhecido:** a ação permanece indisponível e o sistema permite repetir a consulta do estado. **Dados inválidos ou sessão indisponível:** o sistema apresenta erro. **Falha na gravação:** o sistema apresenta erro e mantém o estado anterior da estrela.

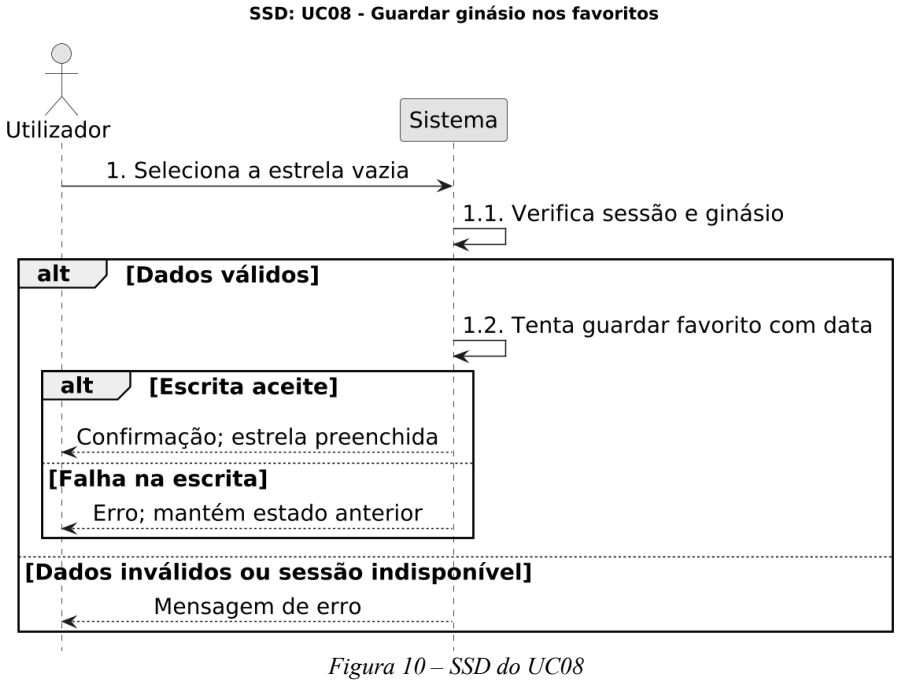


Figura 10 – SSD do UC08

2.3.9. UC09 – Consultar favoritos

Descrição	Consultar os ginásios guardados nos favoritos
Pré-Condição	O utilizador tem sessão iniciada
Pós-Condição	A lista de favoritos é apresentada sem alterar os dados guardados
Caminho Básico	**1.** O utilizador seleciona "Favoritos"; **2.** O sistema consulta os favoritos do utilizador; **3.** O sistema ordena os resultados por data de adição, do mais recente para o mais antigo; **4.** O sistema apresenta a lista.
Caminhos alternativos	**Lista vazia:** o sistema apresenta uma mensagem informativa. **Falha na consulta:** o sistema apresenta erro e permite repetir. **Seleção de um favorito:** o sistema abre a ficha do ginásio através do UC04.

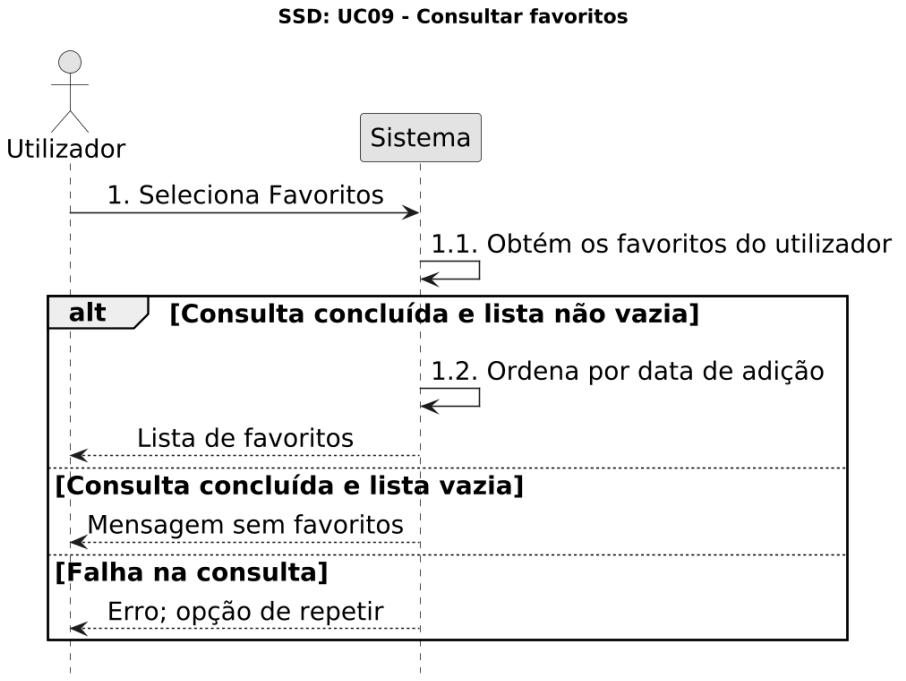


Figura 11 – SSD do UC09

2.3.10. UC10 – Remover ginásio dos favoritos

Descrição	Remover um ginásio dos favoritos do utilizador
Pré-Condição	O utilizador tem sessão iniciada e o ginásio encontra-se nos seus favoritos
Pós-Condição	O favorito é eliminado e a interface é atualizada
Caminho Básico	**1.** O utilizador seleciona a estrela preenchida na ficha ou o botão de remoção na lista; **2.** O sistema verifica a sessão e identifica o favorito; **3.** O sistema elimina o favorito; **4.** O sistema confirma a remoção e atualiza a estrela e a lista.
Caminhos alternativos	**Dados inválidos ou sessão indisponível:** o sistema apresenta erro. **Falha na eliminação:** o sistema apresenta uma mensagem e permite uma nova tentativa.

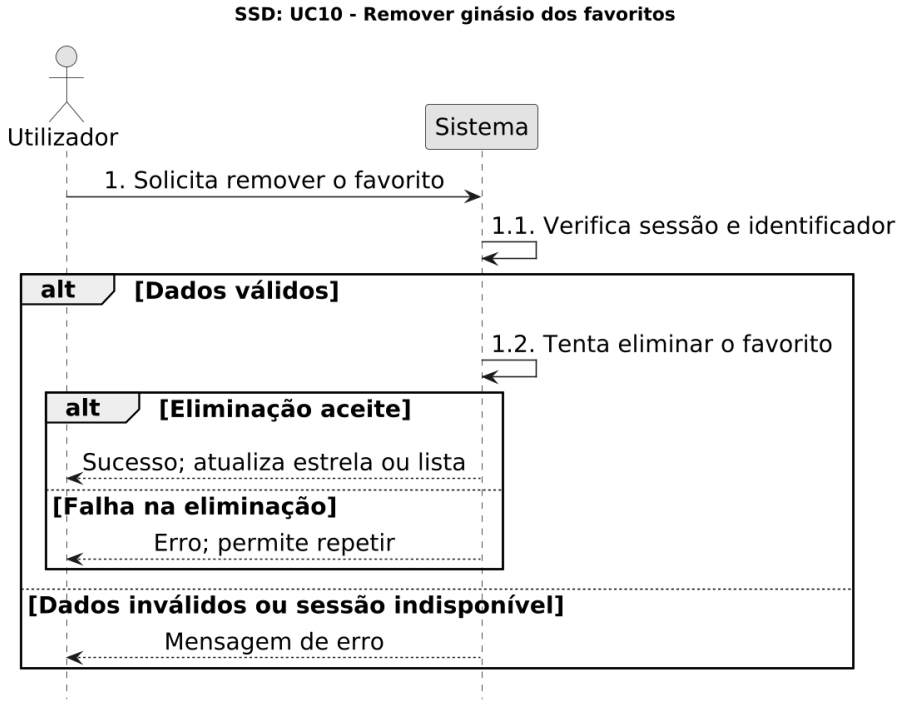


Figura 12 – SSD do UC10

2.3.11. UC11 – Consultar fotografias do ginásio

Descrição	Visualizar e percorrer as fotografias do ginásio
Pré-Condição	A ficha do ginásio encontra-se aberta
Pós-Condição	As fotografias disponíveis são apresentadas sem alterações de dados
Caminho Básico	**1.** O utilizador consulta a área de fotografias da ficha; **2.** O sistema verifica as referências disponíveis e descarrega a imagem; **3.** O sistema apresenta a fotografia; **4.** Quando existem várias fotografias, o utilizador desliza para as percorrer; **5.** O sistema atualiza a imagem e o indicador de posição.
Caminhos alternativos	**Sem fotografias disponíveis:** o sistema apresenta uma indicação de ausência de fotografias. **Falha no descarregamento:** a imagem pode permanecer vazia; a implementação atual não apresenta uma mensagem específica.

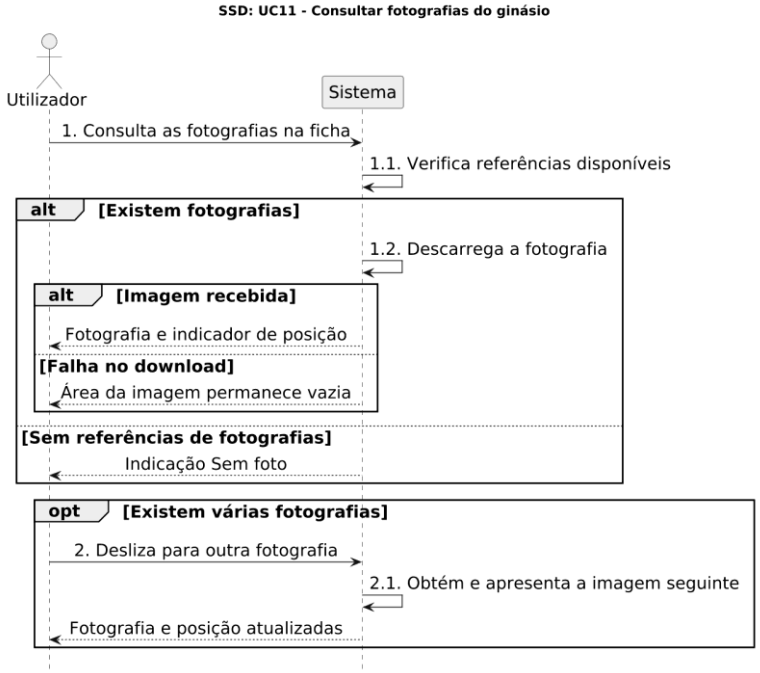


Figura 13 – SSD do UC11

2.3.12. UC12 – Consultar avaliações públicas do ginásio

Descrição	Consultar as avaliações públicas disponibilizadas pelo Google Places
Pré-Condição	A ficha do ginásio foi carregada
Pós-Condição	As avaliações disponíveis são apresentadas, sem criação ou alteração de avaliações
Caminho Básico	**1.** O utilizador consulta a zona de avaliações da ficha; **2.** O sistema verifica as avaliações recebidas com os detalhes do ginásio; **3.** O sistema prepara o nome do autor, a nota e o texto; **4.** O sistema apresenta a lista de avaliações públicas.
Caminhos alternativos	**Nenhuma avaliação devolvida:** o sistema oculta a lista. **Falha ao obter a ficha:** aplica-se o fluxo alternativo do UC04.

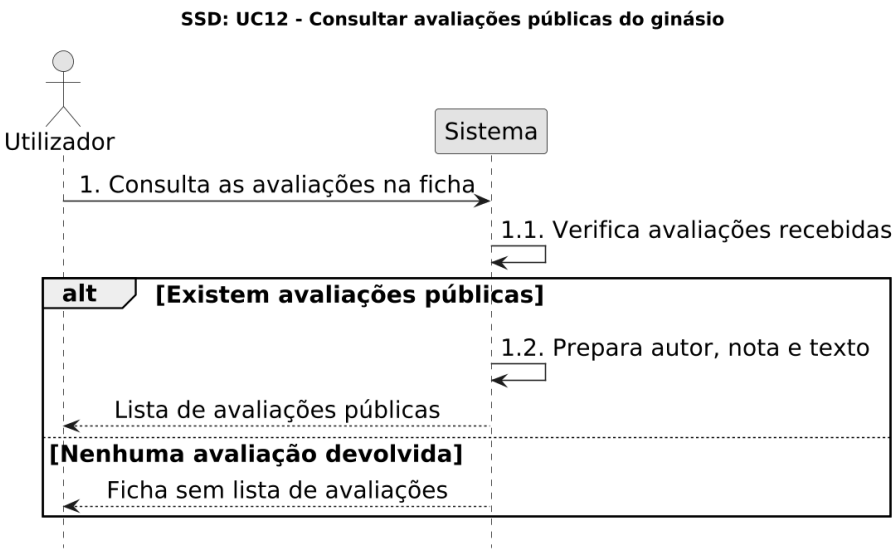


Figura 14 – SSD do UC12

2.3.13. UC13 – Registar visita por QR Code

Descrição	Registar uma visita ao ginásio através da leitura de um QR Code com a câmara
Pré-Condição	O utilizador tem sessão iniciada e perfil existente; a ficha está carregada e o ginásio encontra-se identificado como aberto
Pós-Condição	Uma nova visita é acrescentada ao histórico, com o ginásio e a data e hora do registo
Caminho Básico	**1.** O utilizador seleciona "Registar Visita"; **2.** O sistema verifica a sessão, o ginásio e o estado de abertura; **3.** O utilizador concede permissão da câmara, quando necessário, e lê o QR Code; **4.** O sistema valida o conteúdo lido; **5.** O sistema obtém o histórico e guarda a nova visita; **6.** O sistema apresenta uma confirmação de sucesso.
Caminhos alternativos	**Ginásio fechado, dados inválidos ou sessão indisponível:** o sistema recusa o registo. **Permissão de câmara recusada ou leitura cancelada:** a visita não é registada. **QR Code não aceite:** o sistema apresenta erro. **Falha na consulta ou gravação do histórico:** o sistema apresenta erro e permite tentar novamente.

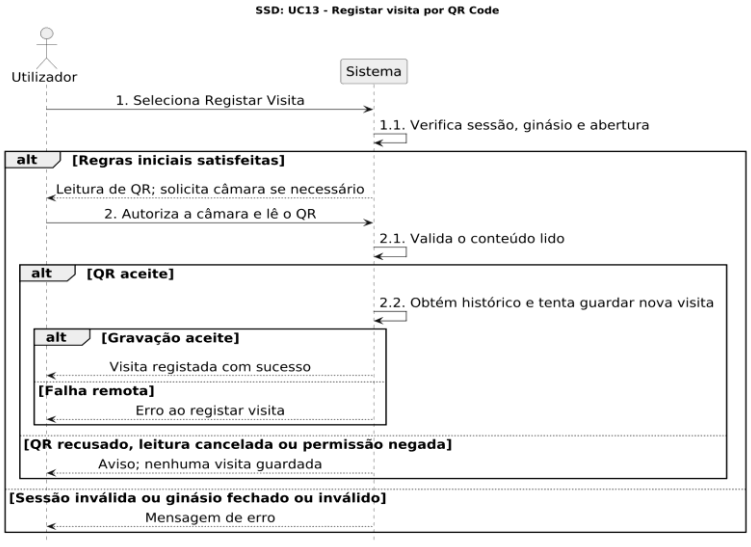


Figura 15 – SSD do UC13

2.3.14. UC14 – Consultar histórico de visitas

Descrição	Consultar os registos de visitas efetuadas pelo utilizador
Pré-Condição	O utilizador tem sessão iniciada
Pós-Condição	O histórico é apresentado sem alteração dos registos
Caminho Básico	**1.** O utilizador seleciona "Ginásios" na navegação inferior ou o cartão de visitas do perfil; **2.** O sistema obtém o histórico associado ao perfil; **3.** O sistema ordena as visitas da mais recente para a mais antiga; **4.** O sistema apresenta o nome do ginásio e a data e hora de cada visita.
Caminhos alternativos	**Histórico vazio:** o sistema apresenta uma mensagem informativa. **Falha na consulta:** o sistema apresenta erro e um botão para repetir. **Seleção de uma visita:** o sistema abre a ficha atualizada do ginásio através do UC04.

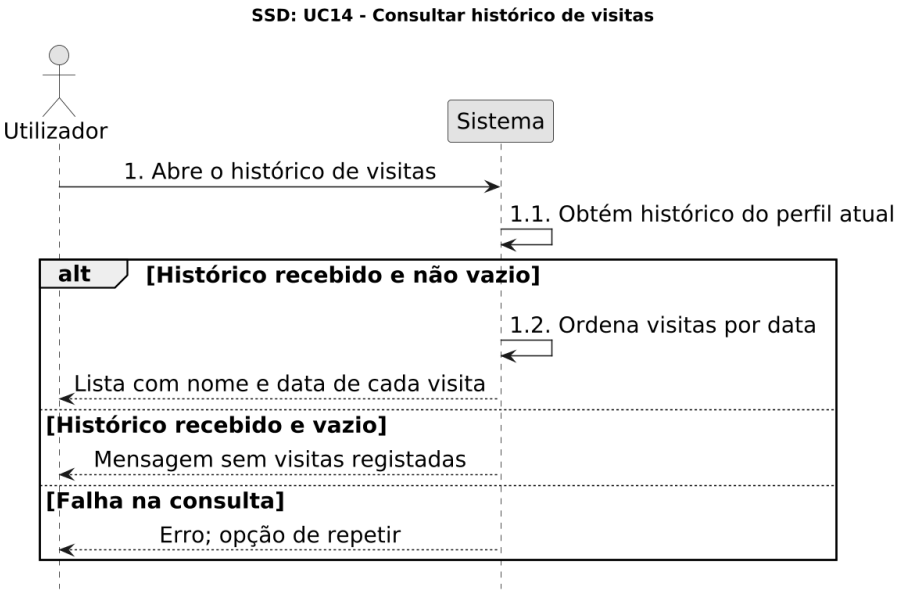


Figura 16 – SSD do UC14

3. Design

3.1. Diagrama de implantação

A aplicação BetterGym é executada num dispositivo Android e comunica por HTTPS com dois serviços externos. O Firebase trata da autenticação e do armazenamento de dados. A Google Maps Platform fornece o mapa e os dados dos ginásios através da Google Places API.

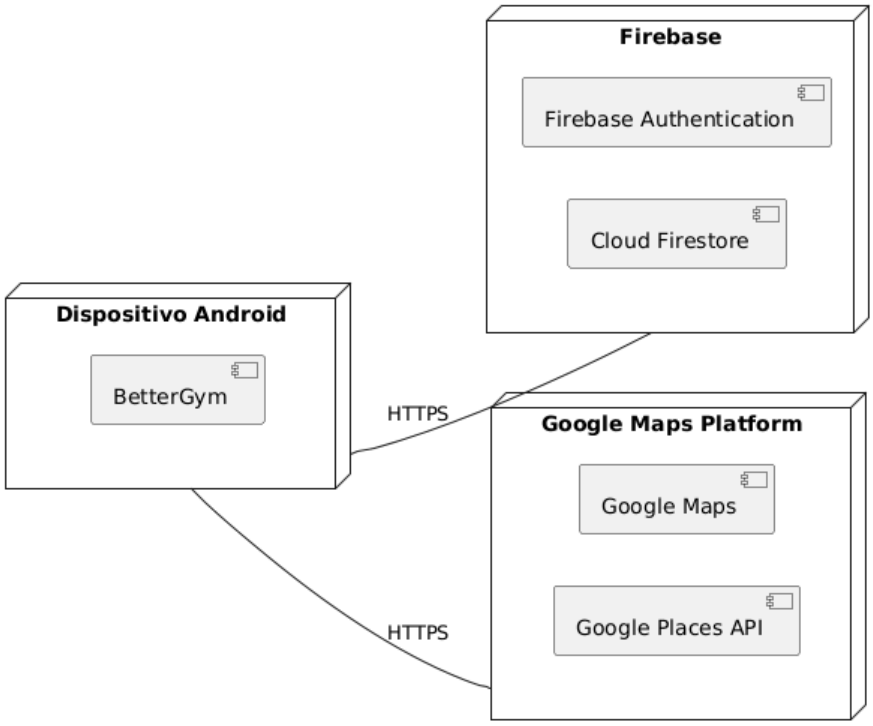


Figura 17 – Diagrama de implantação

3.2. Diagrama de componentes

O diagrama de componentes representa a organização interna da aplicação e das dependências entre os seus componentes. A interface de utilizador recorre ao service para executar as operações da aplicação e aos adapters para apresentar os dados. O service utiliza o model, os DTO's e o mapper para representar e converter os dados. Os handlers processam os dados em JSON e gerem a comunicação com as API's externas por HTTPS.

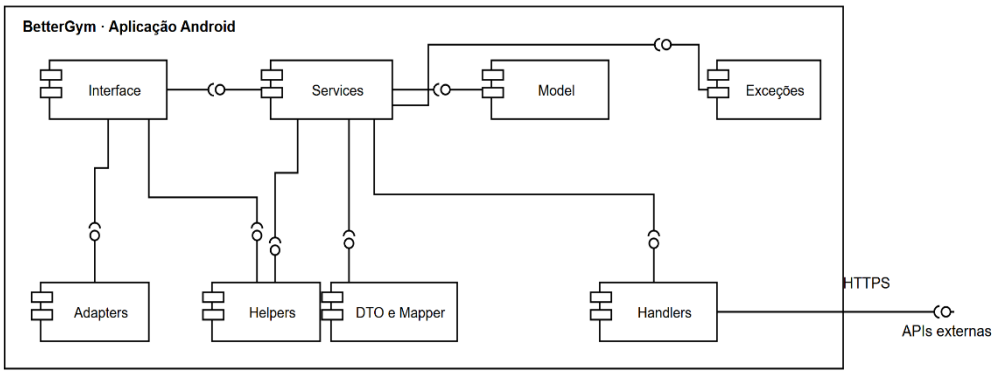


Figura 18 - Diagrama de componentes

3.3. Diagrama de pacotes

O diagrama de pacotes apresenta a organização do código dentro do pacote com.example.bettergym. O pacote views reúne os ecrãs e os adaptadores da aplicação e recorre ao pacote service para executar as operações pedidas pelo utilizador. O pacote model representa os dados da aplicação, dto reúne as classes de transferência e conversão de dados e handler trata da comunicação HTTP e do processamento de JSON. Os pacotes helper e exceptions reúnem, respetivamente, funções auxiliares e exceções da aplicação. As setas tracejadas apontam do pacote que utiliza para o pacote utilizado.

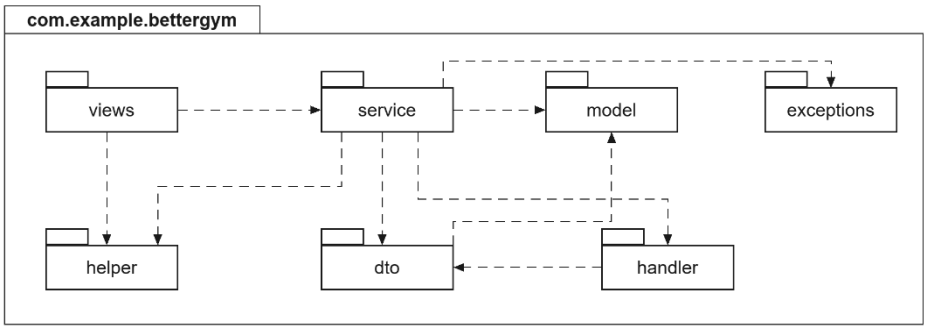


Figura 19 – Diagrama de pacotes

3.4. Diagrama de classes

3.4.1. Model

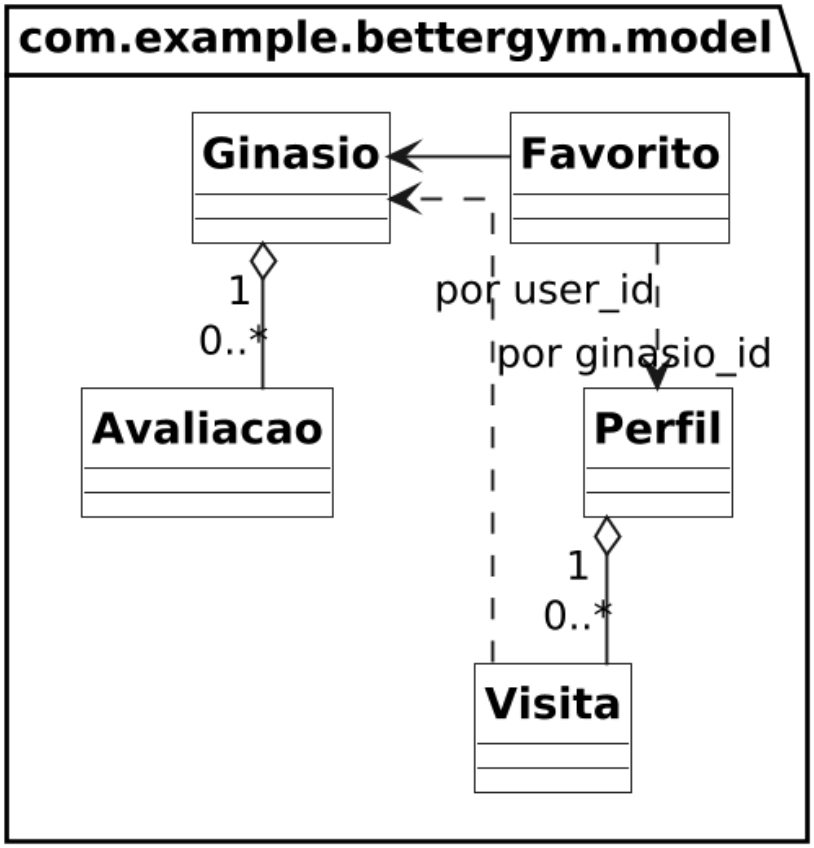


Figura 20 – Diagrama de classes do pacote Model

3.4.2. Views

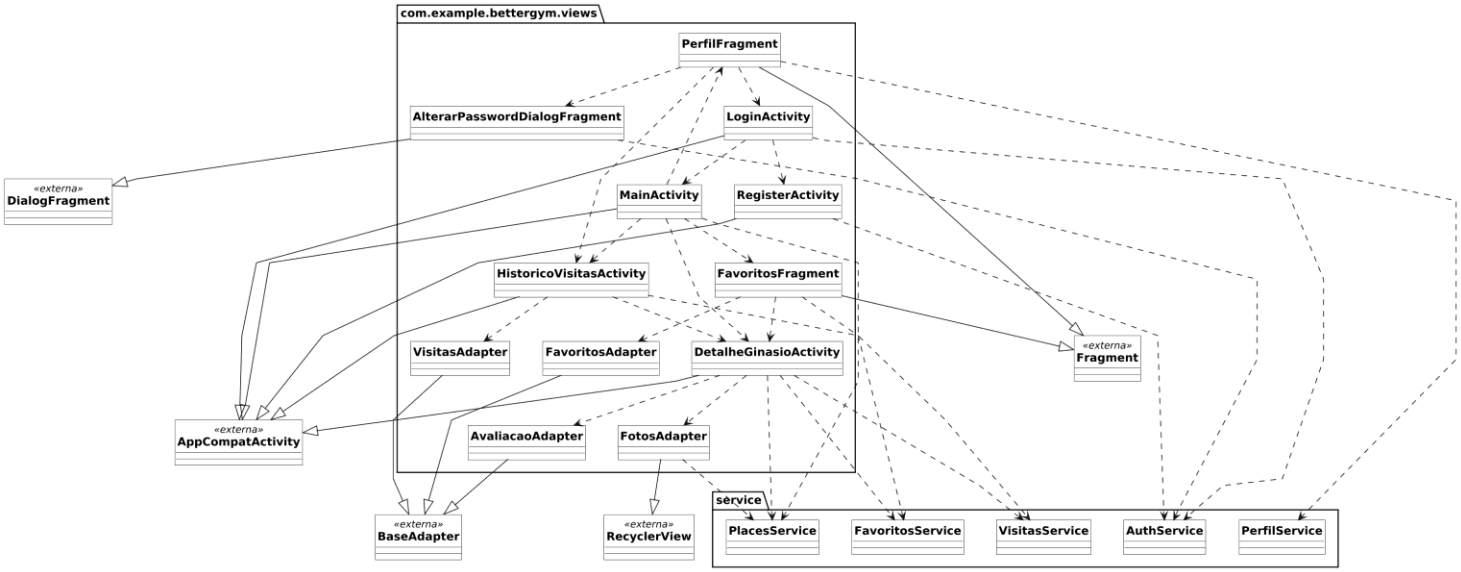


Figura 21 – Diagrama de classes do pacote Views

3.4.3. Service

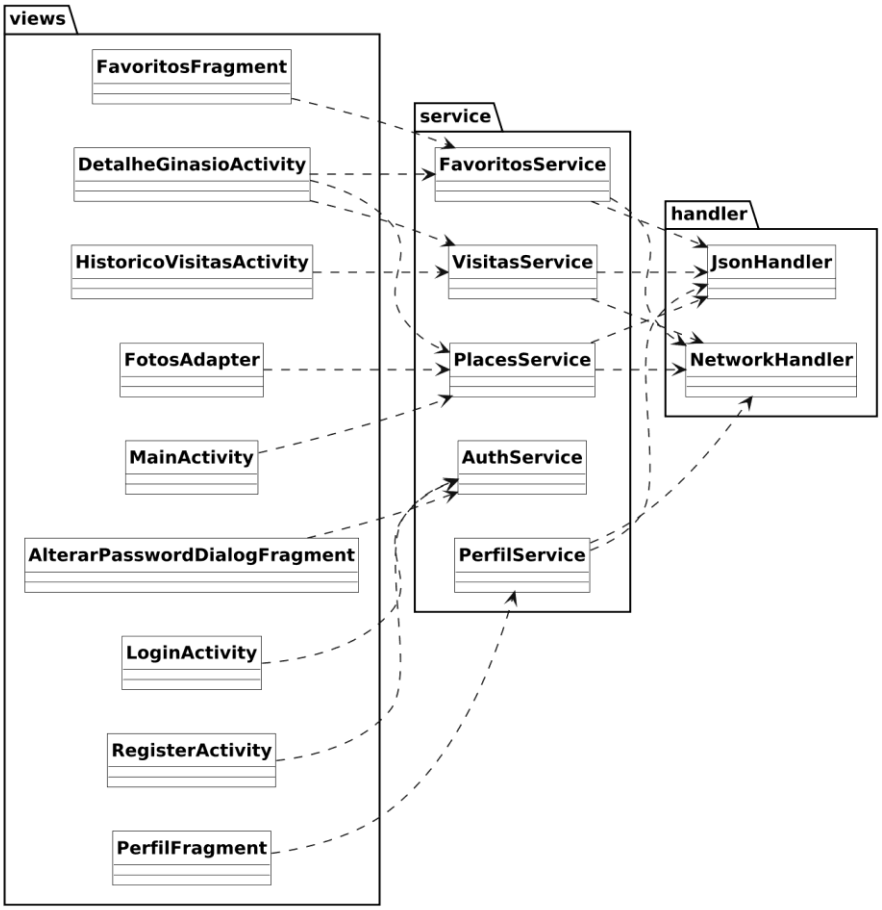


Figura 22 – Diagrama de classe do pacote Service

3.4.4. DTO e Mapper

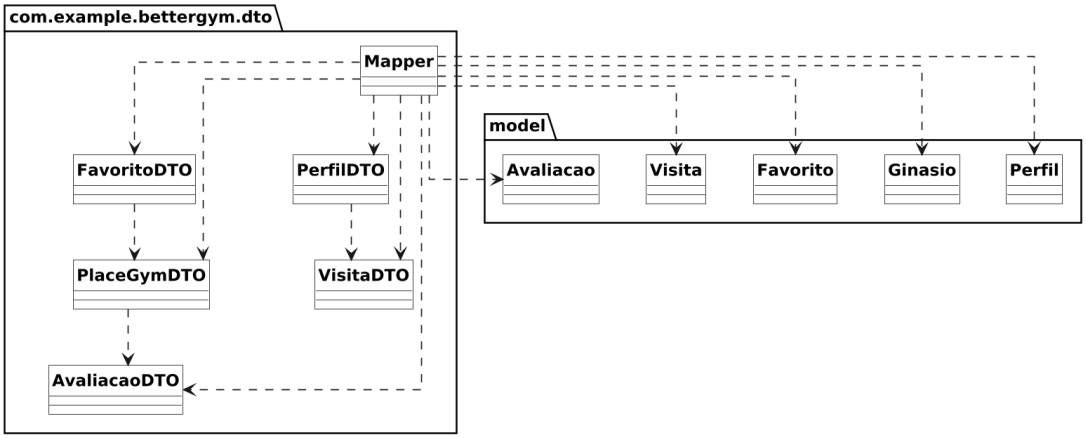


Figura 23 – Diagrama de classes do pacote DTO

3.4.5. Handler

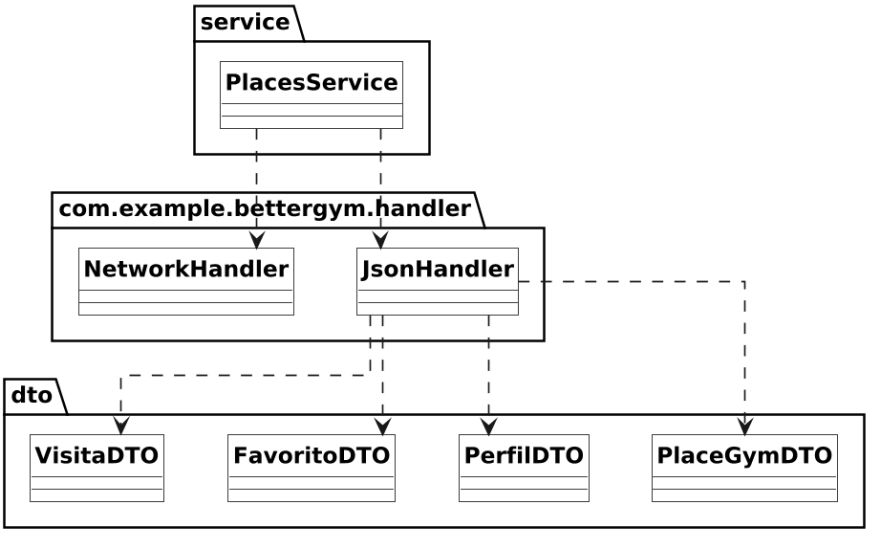


Figura 24 – Diagrama de classes do pacote Handler

3.4.6. Helper

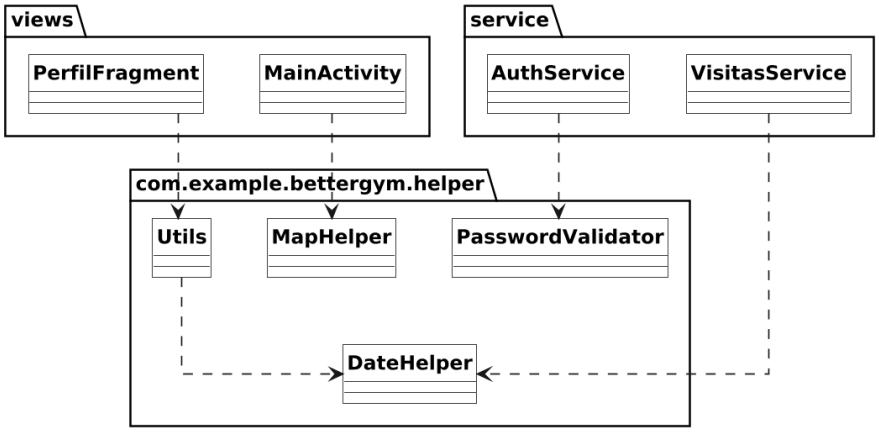


Figura 25 – Diagrama de classes do pacote Helper

3.4.7. Exceptions

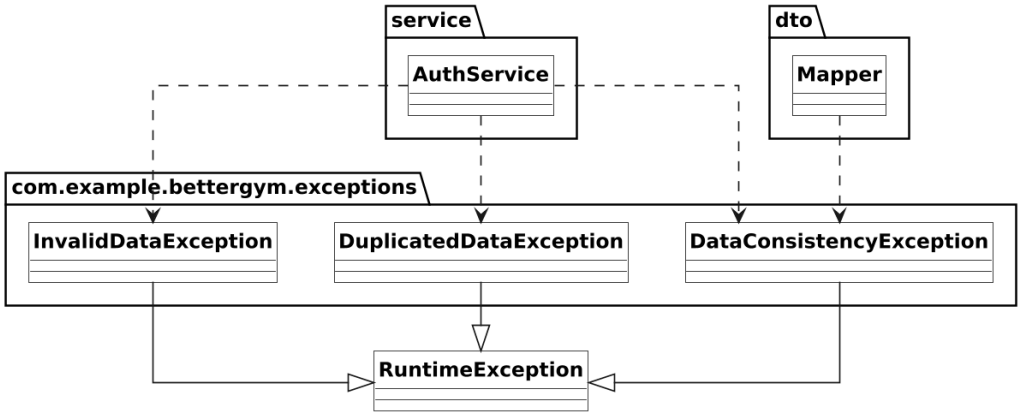


Figura 26 – Diagrama de classes do pacote Exceptions

3.5. Diagrama de sequência

3.5.1. UC01 – Registar conta

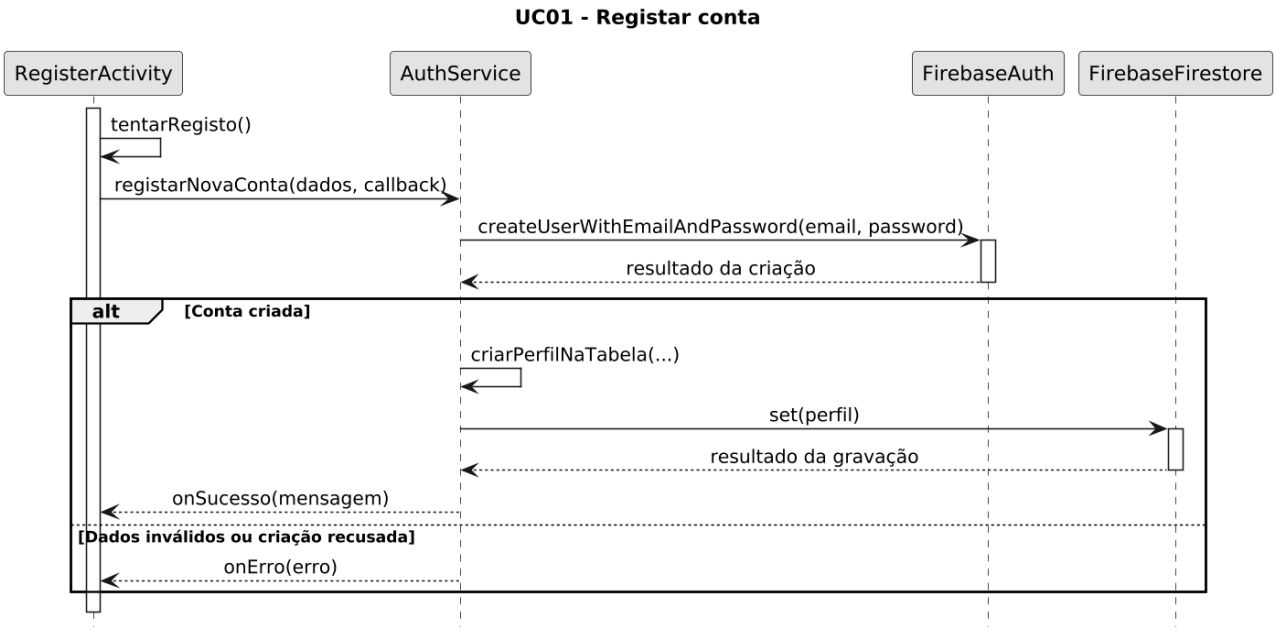


Figura 27 – Diagrama de sequência da UC01

3.5.2. UC02 – Iniciar sessão

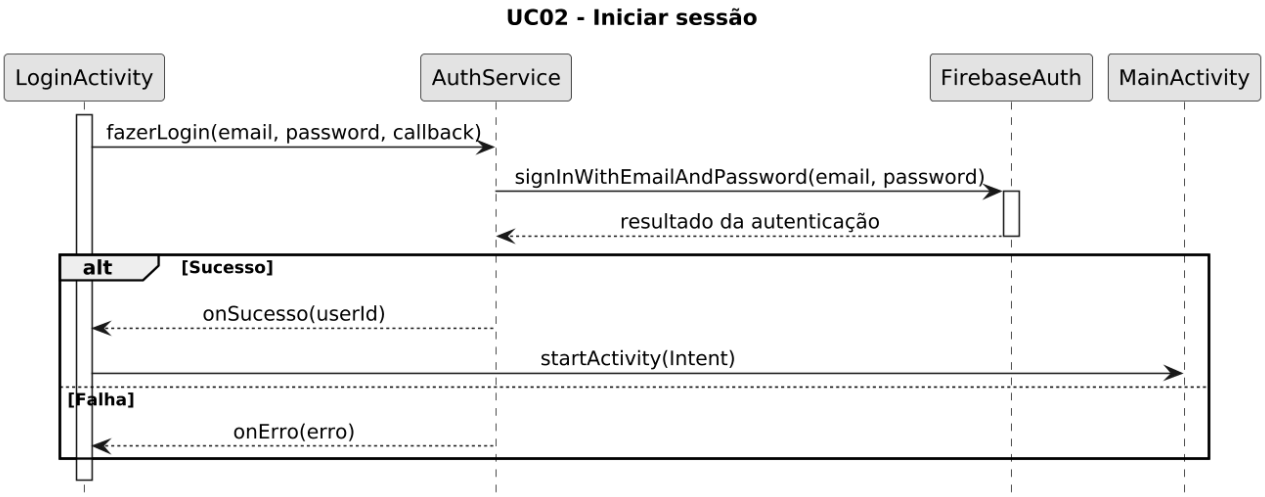


Figura 28 – Diagrama de sequência da UC02

3.5.3. UC03 – Descobrir ginásios

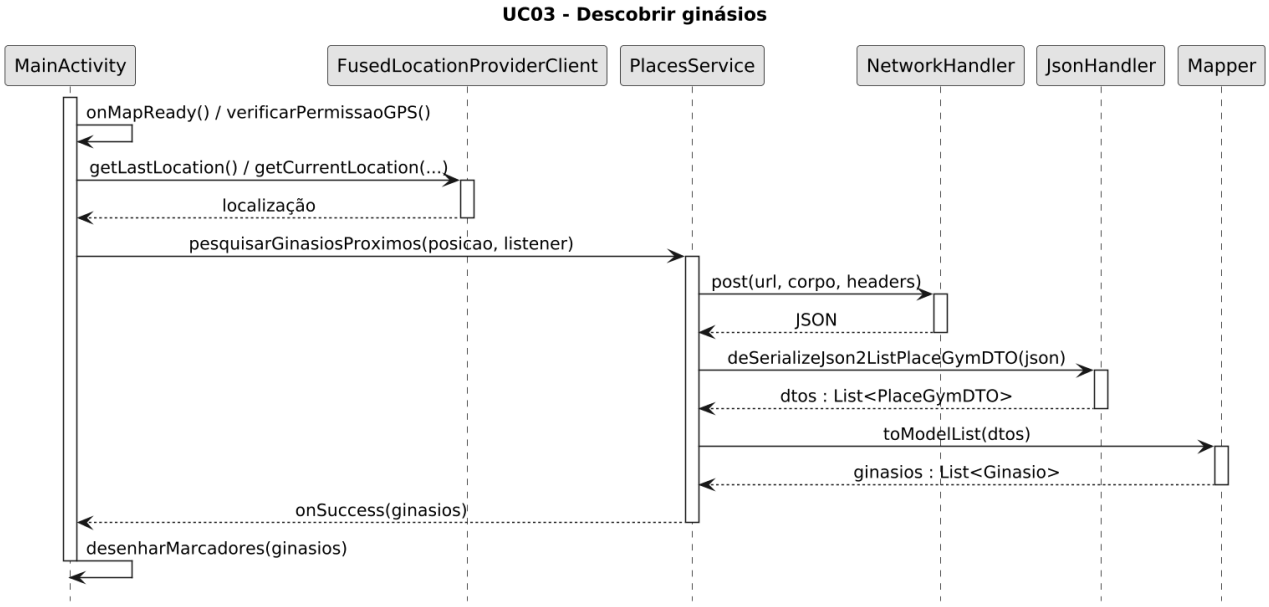


Figura 29 – Diagrama de sequência da UC03

3.5.4. UC04 – Consultar ficha do ginásio

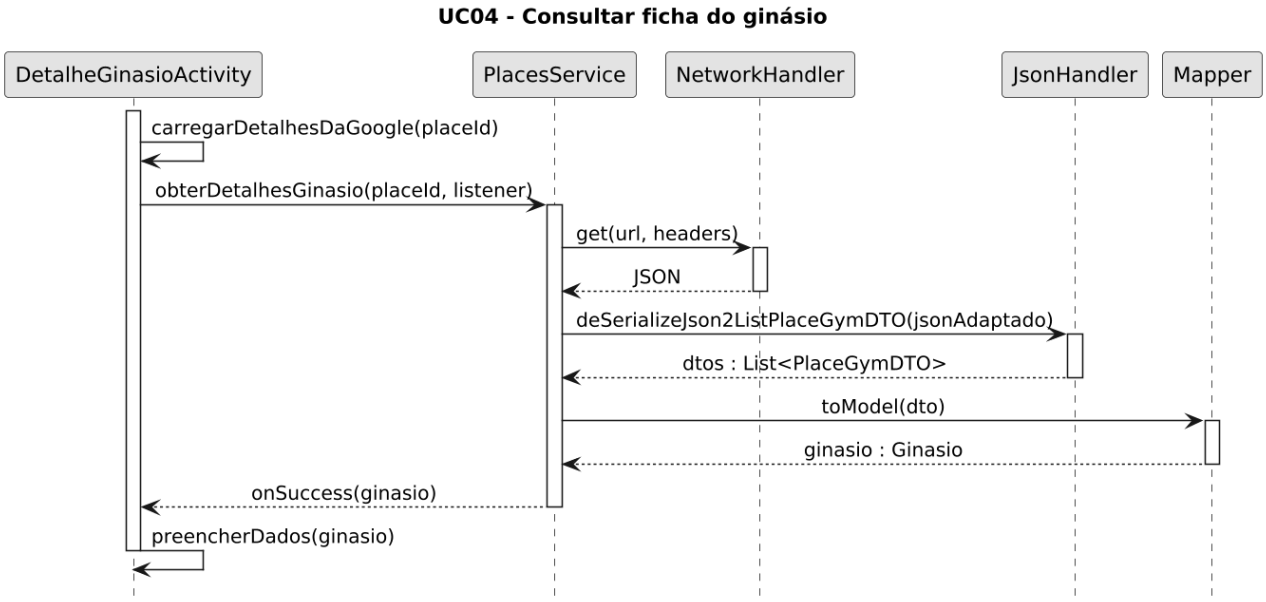


Figura 30 – Diagrama de sequência da UC04

3.5.5. UC05 – Consultar perfil

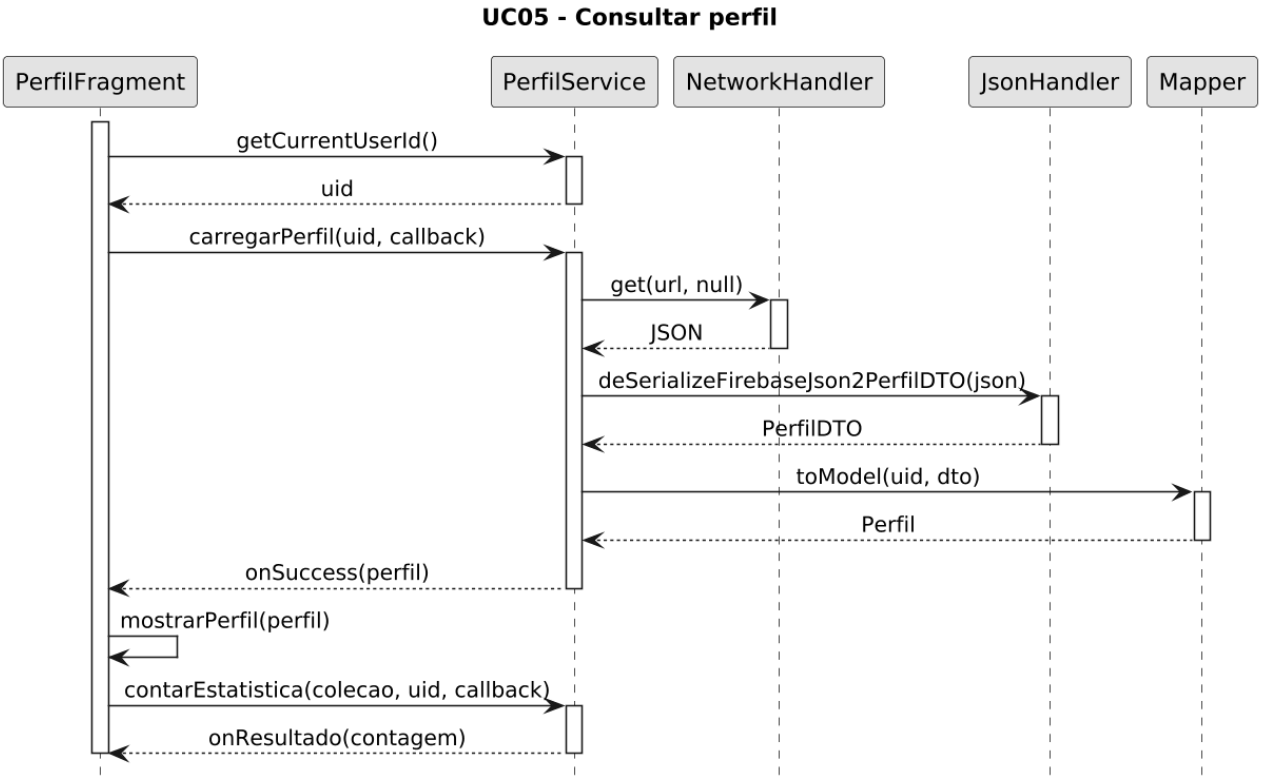


Figura 31 – Diagrama de sequência da UC05

3.5.6. UC06 – Alterar palavra-passe

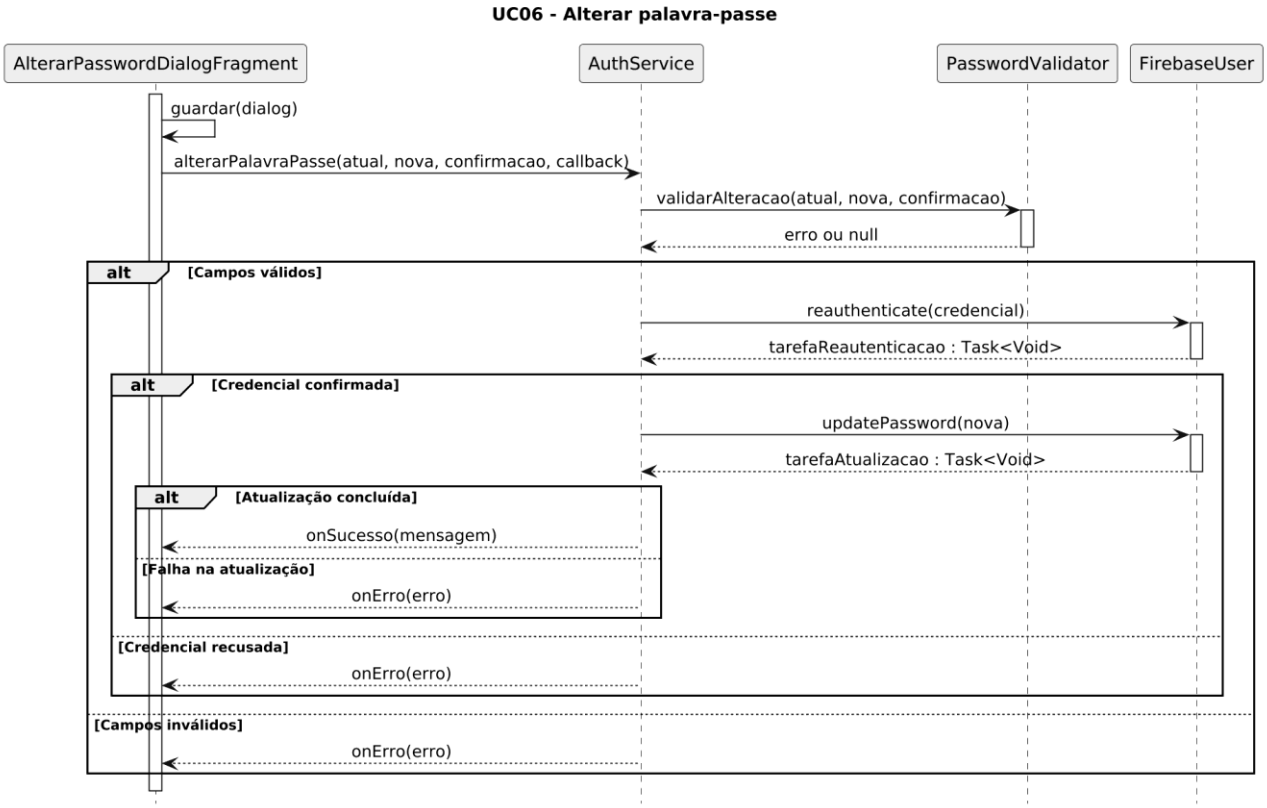


Figura 32 – Diagrama de sequência do UC06

3.5.7. UC07 – Terminar sessão

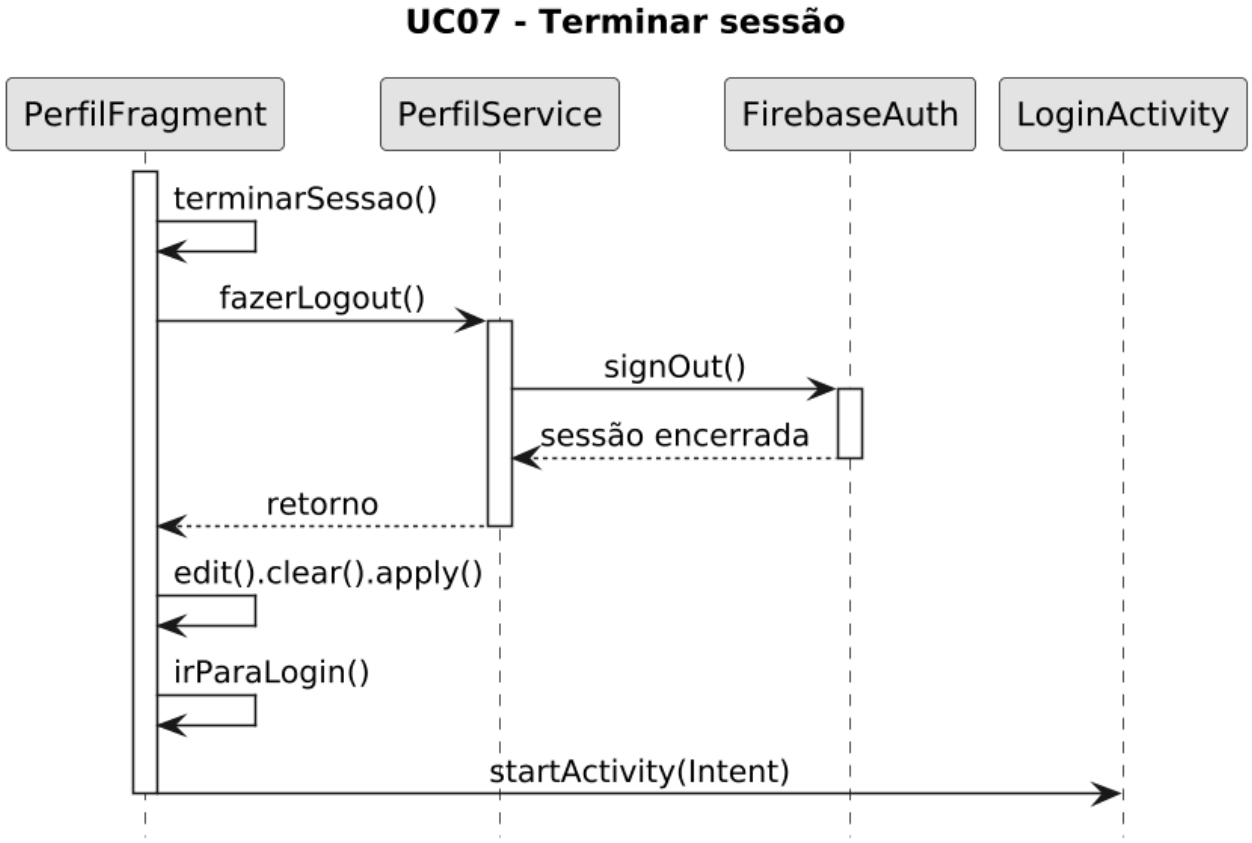


Figura 33 – Diagrama de sequência da UC07

3.5.8. UC08 – Guardar ginásios nos favoritos

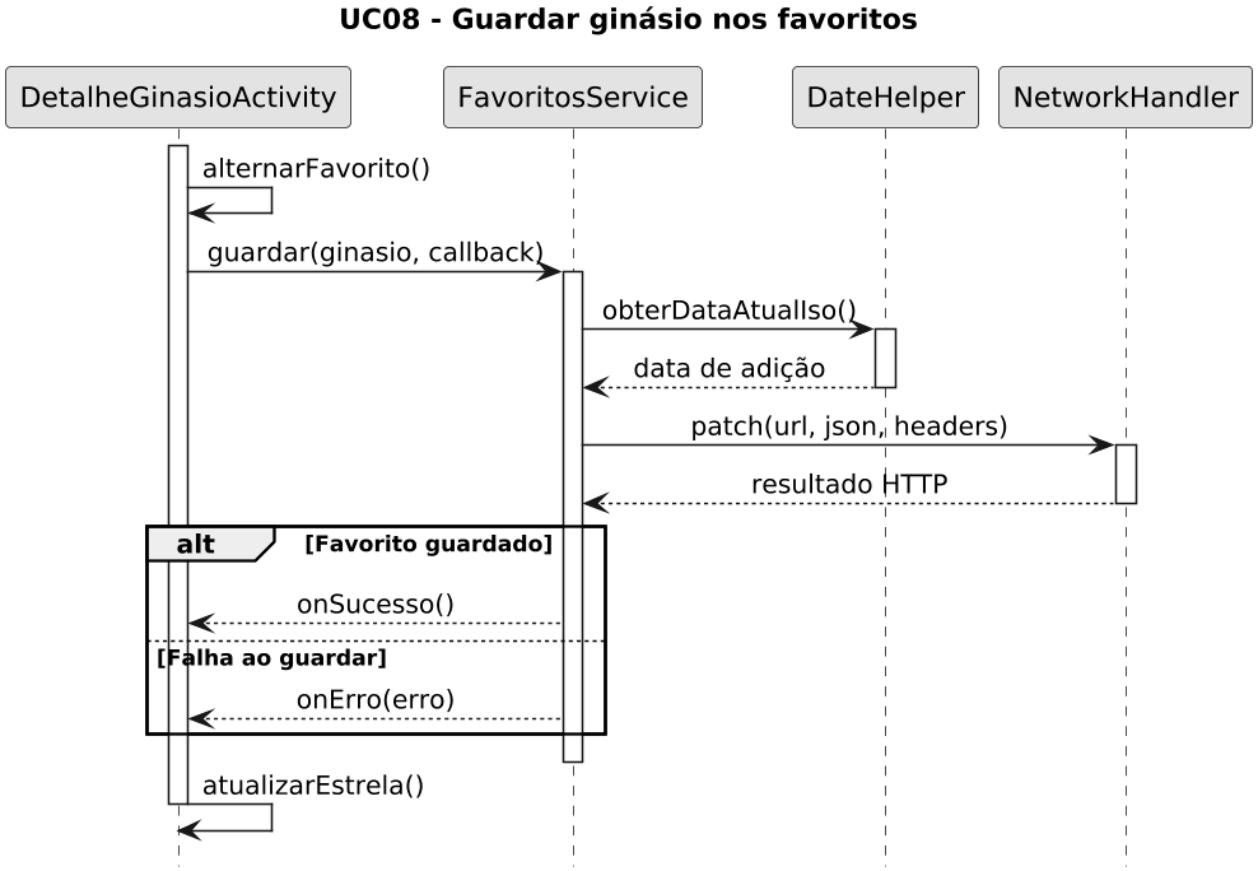


Figura 34 – Diagrama de sequência da UC08

3.5.9. UC09 – Consultar favoritos

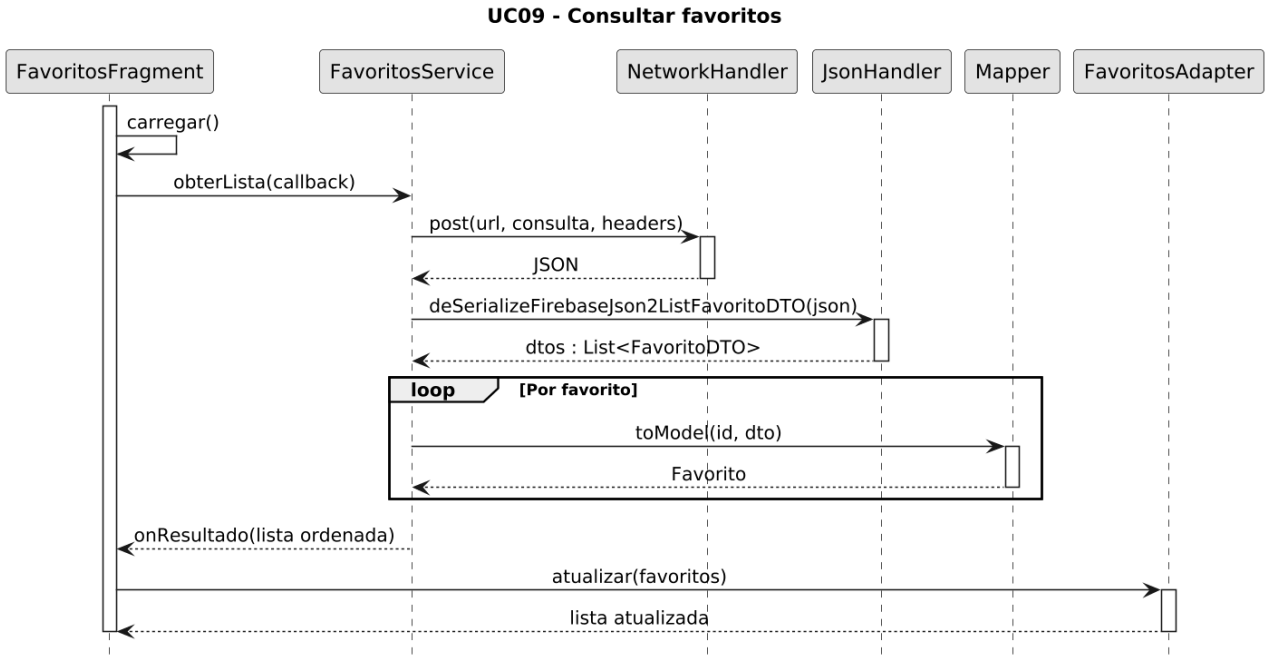


Figura 35 – Diagrama de sequência da UC09

3.5.10. UC10 – Remover ginásio dos favoritos

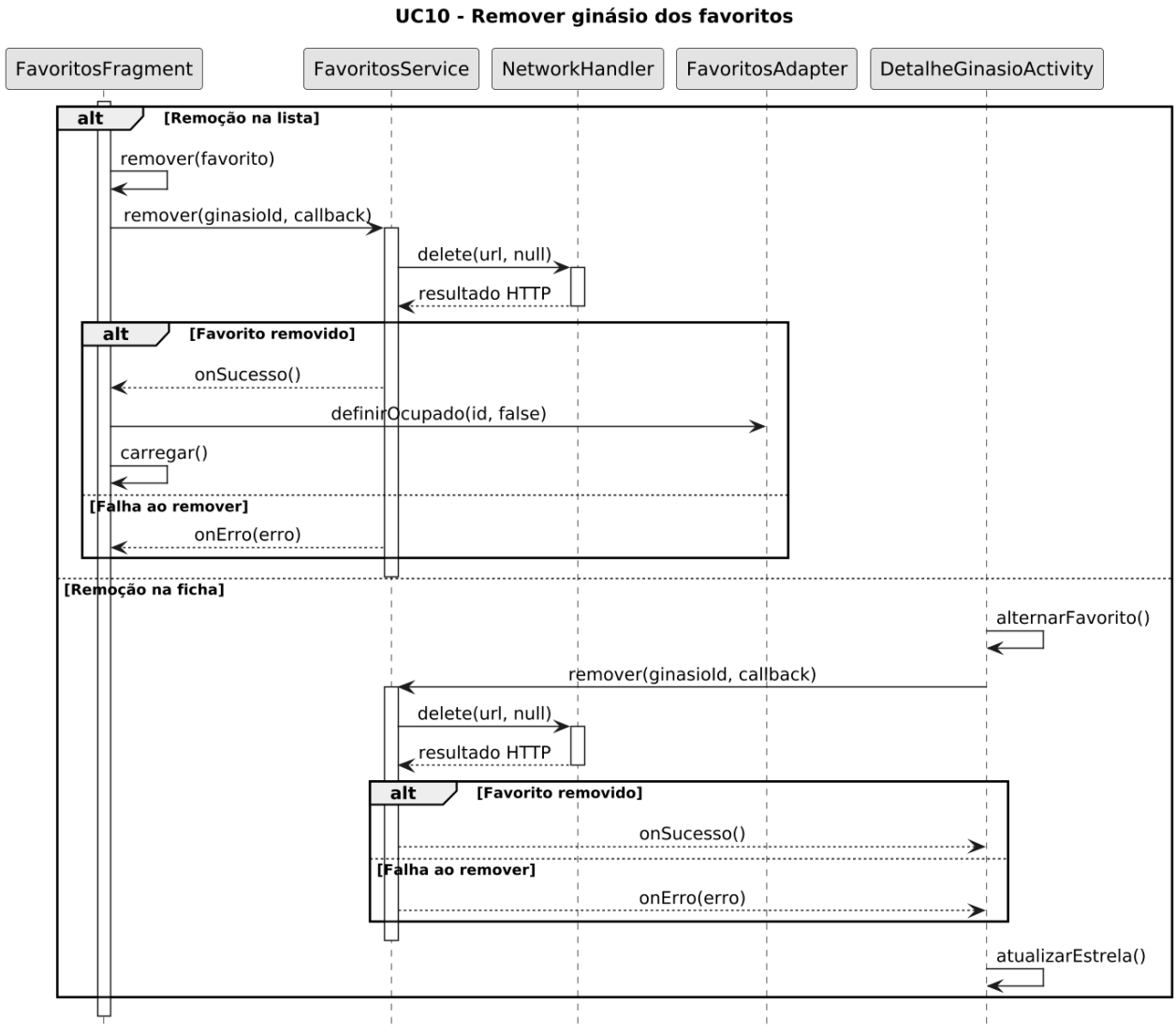


Figura 36 – Diagrama de sequência da UC10

3.5.11. UC11 – Consultar fotografias do ginásio

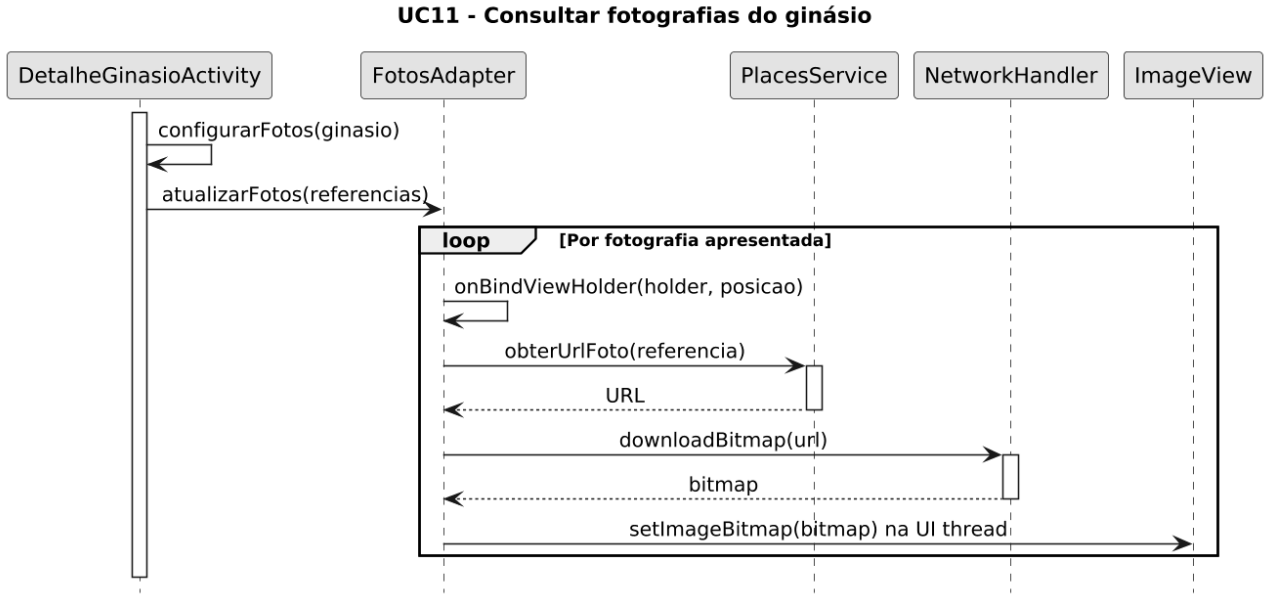


Figura 37 - Diagrama de sequência da UC11

3.5.12. UC12 – Consultar avaliações públicas do ginásio

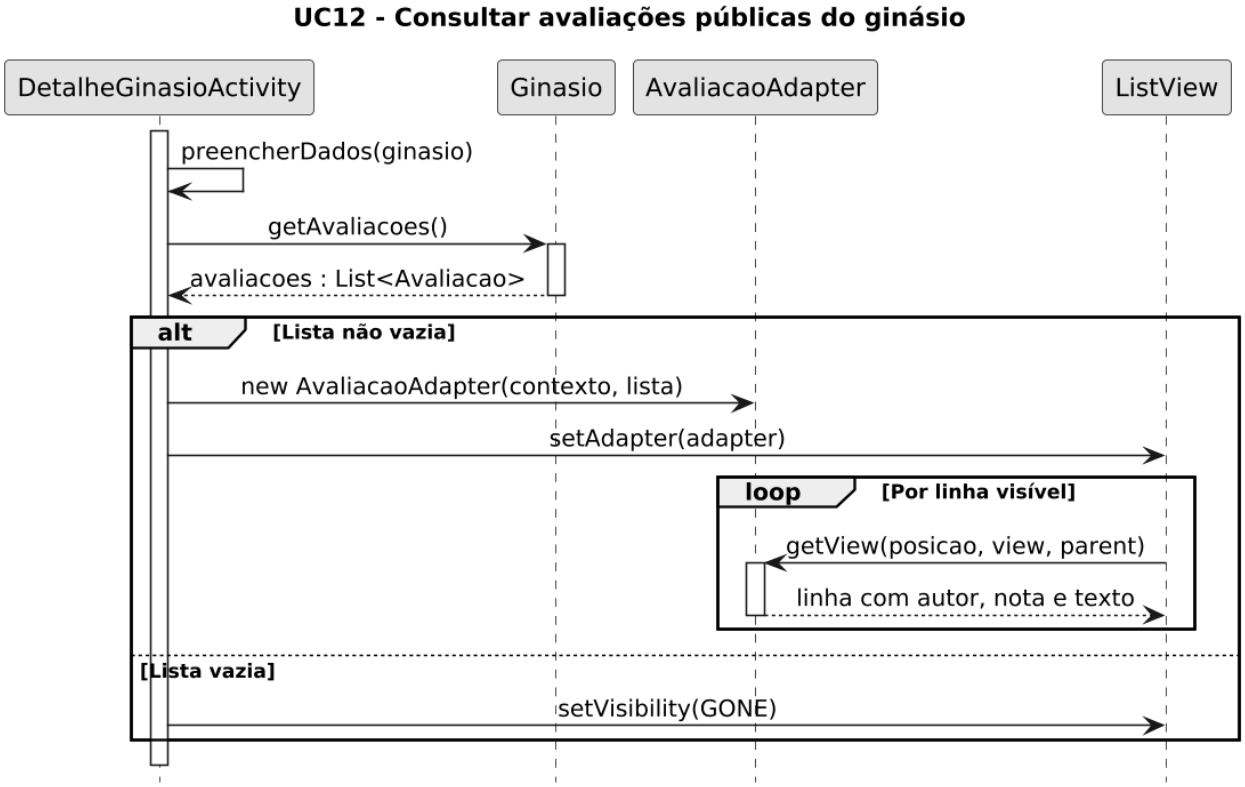


Figura 38 - Diagrama de sequência da UC12

3.5.13. UC13 – Registar visita por QR Code

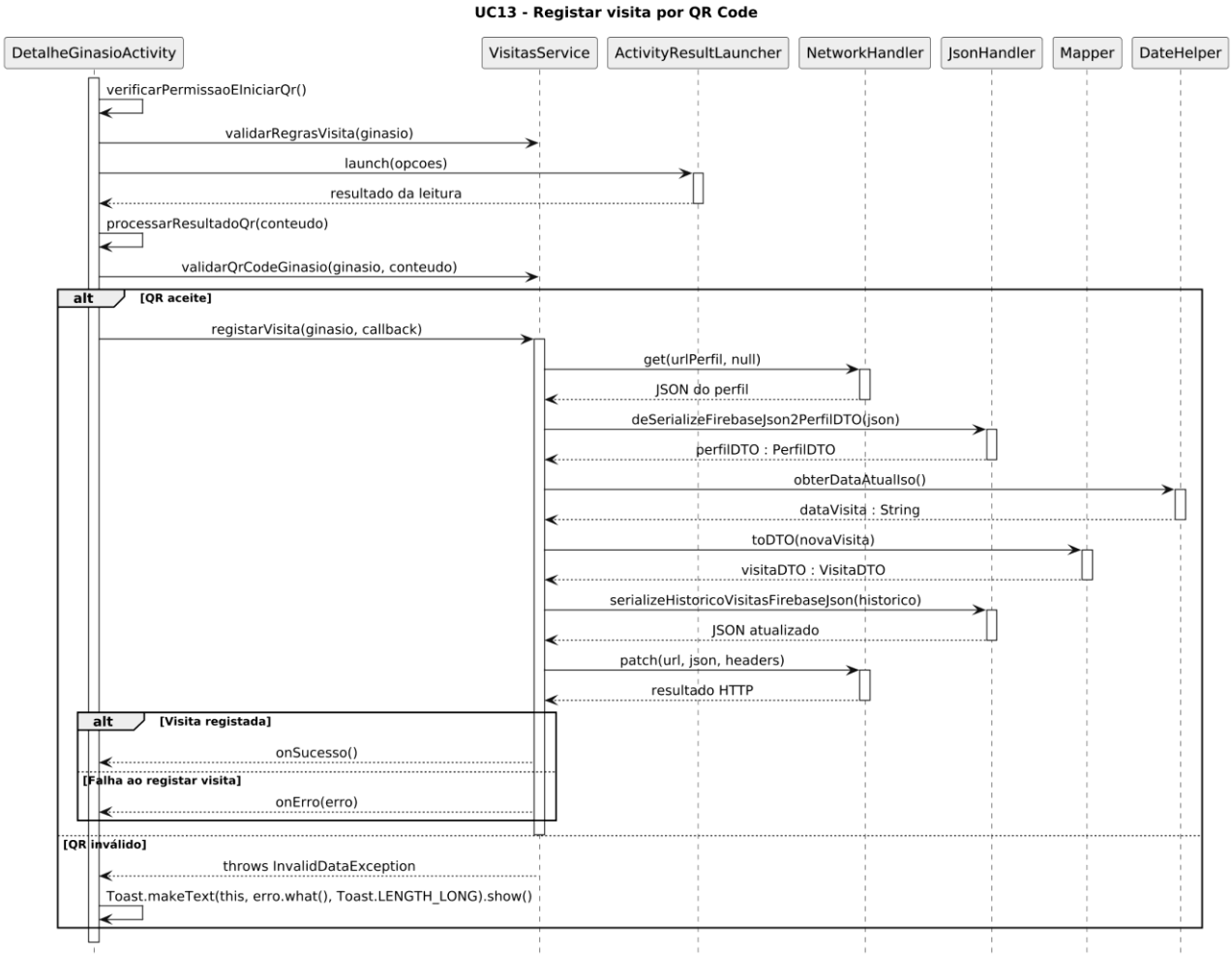


Figura 39 - Diagrama de sequência da UC13

3.5.14. UC14 – Consultar histórico de visitas

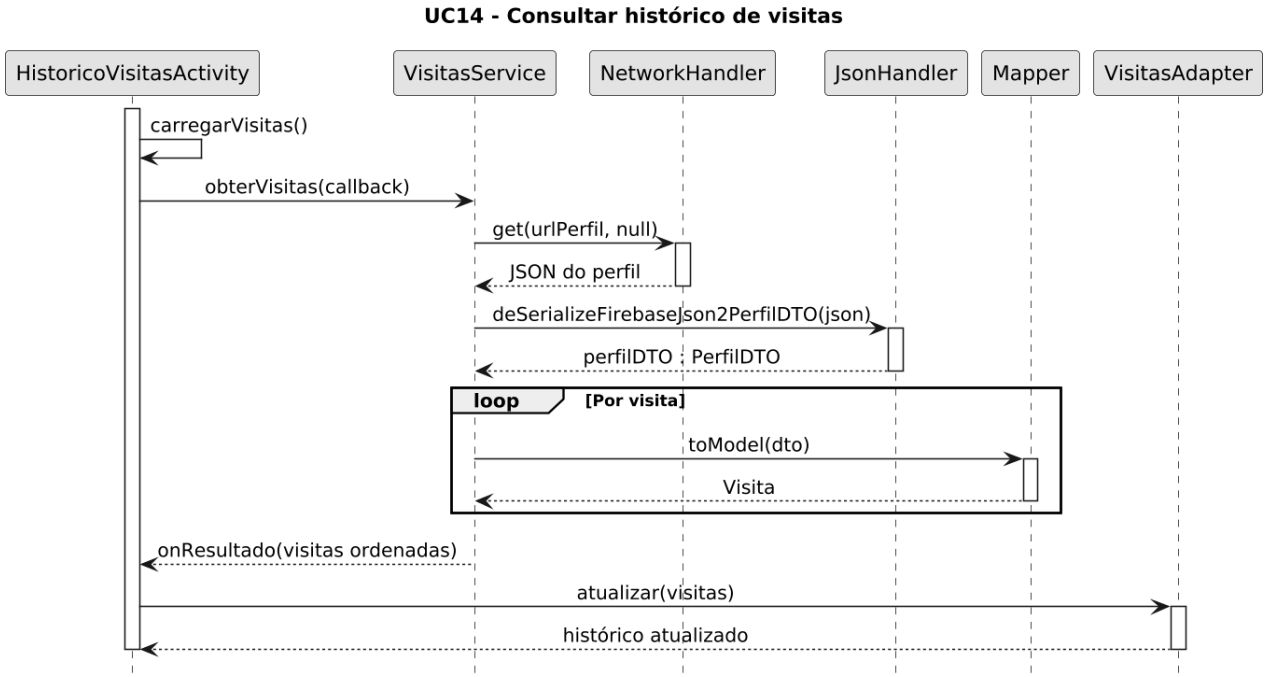


Figura 40 - Diagrama de sequência da UC14

3.6. Mockups

3.6.1. Conta

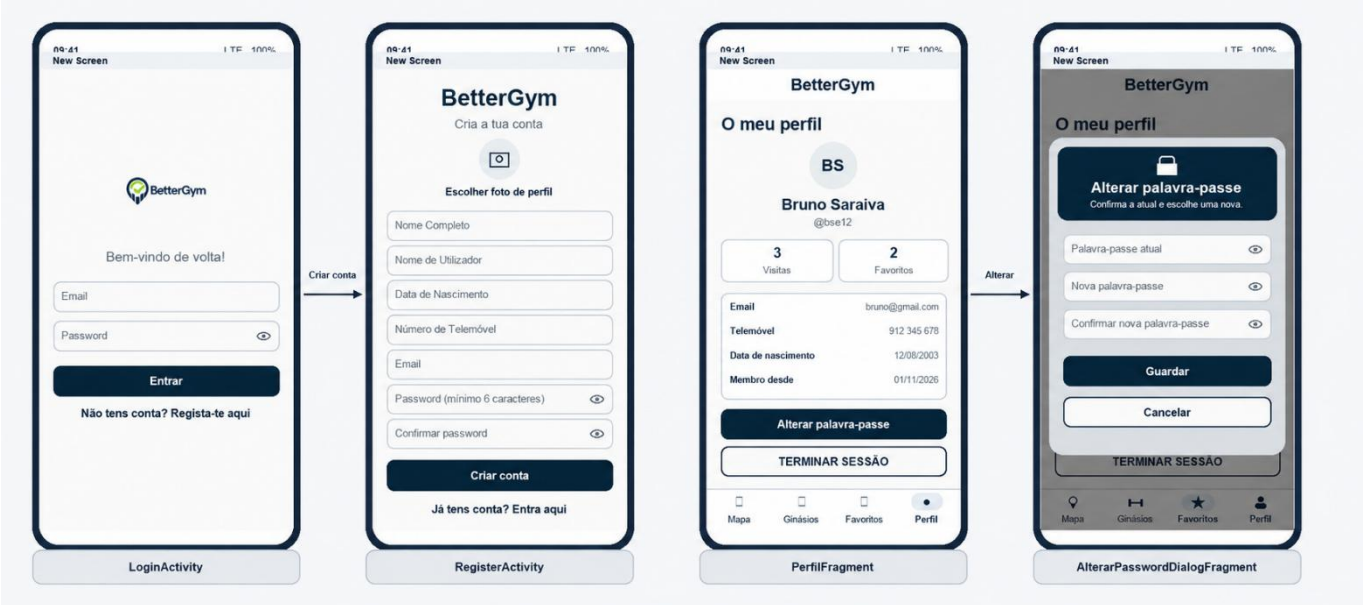


Figura 41 – Mockups de acesso e gestão da conta

3.6.2. Descoberta e favoritos

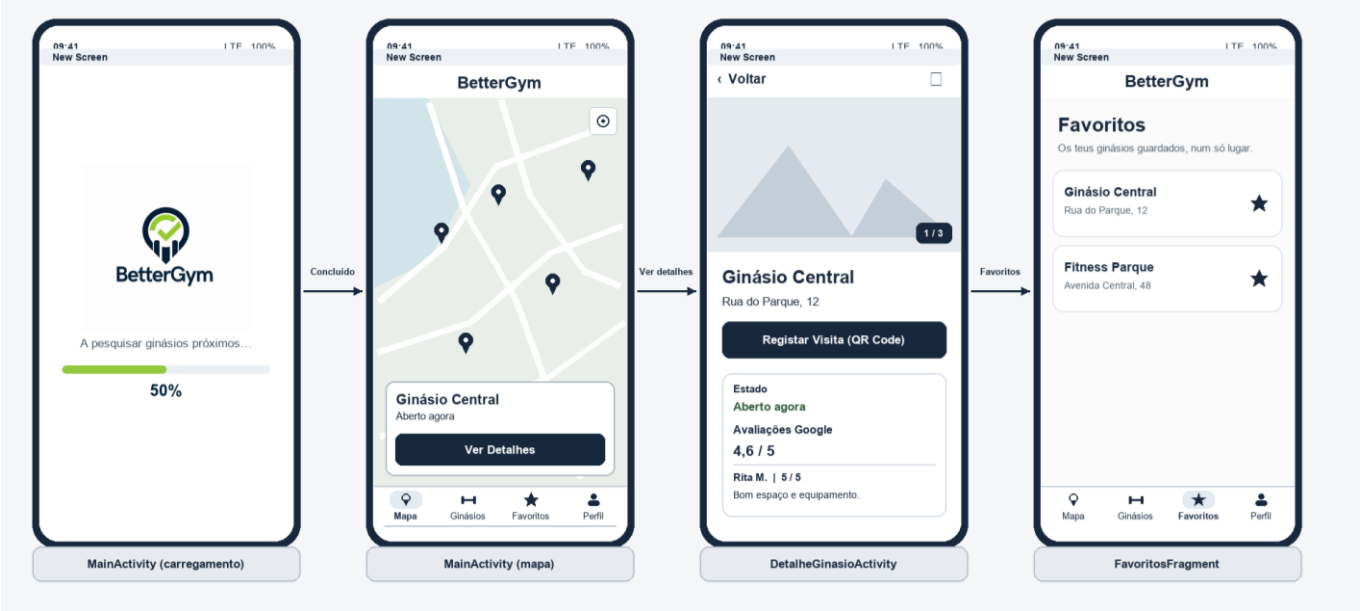


Figura 42 - Mockups dos favoritos e para descoberta

3.6.3. Registo e histórico de visitas

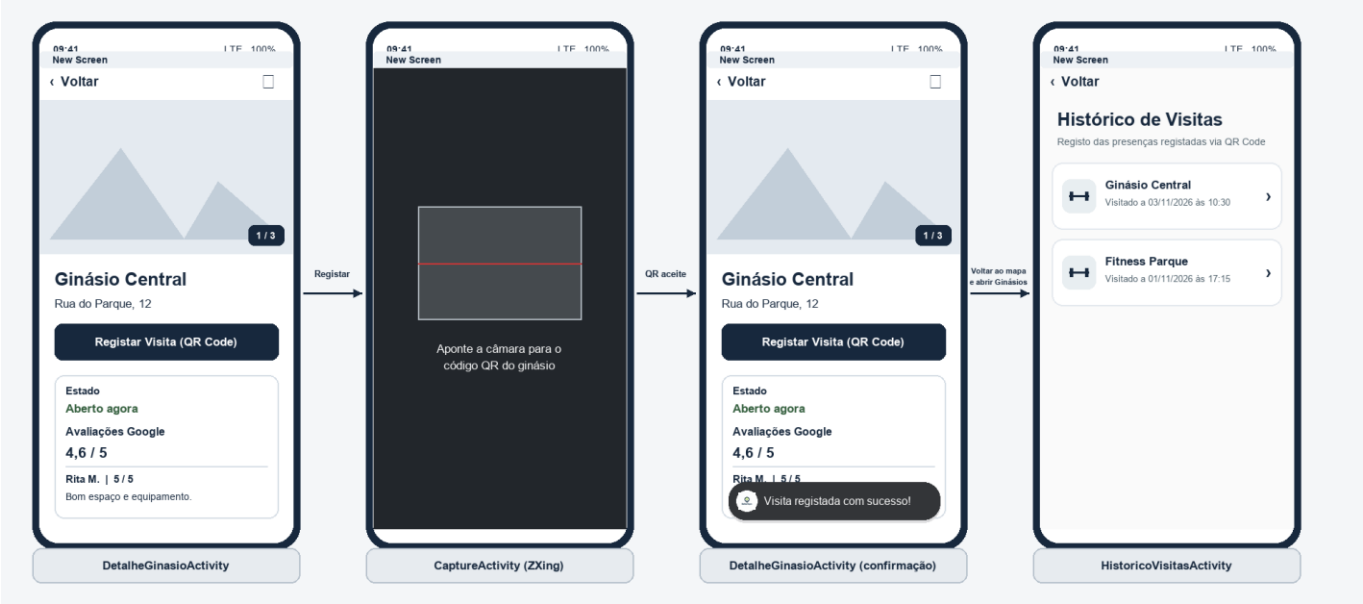


Figura 43 - Mockup de registo e histórico de visitas

4. Implementação

A implementação da BetterGym combina uma interface Android em Java com autenticação, localização, consulta de ginásios e armazenamento dos dados do utilizador. Esta secção apresenta os mecanismos que suportam os vários casos de uso, com destaque para a comunicação HTTP, o tratamento de JSON, a conversão de objetos e a execução de operações assíncronas.

4.1. Organização e convenções

O pacote views contém as Activities, os Fragments e os adapters responsáveis pela apresentação e interação. O pacote service executa as operações de autenticação, perfis, pesquisa de ginásios, favoritos e visitas. NetworkHandler e JsonHandler, no pacote handler, centralizam os pedidos HTTP e a interpretação e construção de JSON. Os DTO transportam os dados recebidos e o Mapper converte-os para as classes de model. Os pacotes helper e exceptions reúnem funções auxiliares e exceções específicas da aplicação.

As classes e interfaces seguem a convenção UpperCamelCase, como AuthService e PerfilCallback, enquanto os métodos e as variáveis seguem, em geral, lowerCamelCase, como fazerLogin e obterVisitas. Os campos guardados na base de dados mantêm nomes com palavras separadas por sublinhados, como user_id e data_visita, para conservar a correspondência entre os documentos e os objetos Java. As constantes utilizam letras maiúsculas e palavras separadas por sublinhados, como EXTRA_GINASIO_ID.

A aplicação utiliza os SDK do Firebase para autenticação e algumas operações no Firestore. Outros acessos são realizados através da API REST do Firestore. Os dados dos ginásios são obtidos através da Google Places API, e o Google Maps apresenta os resultados no mapa. A escolha do mecanismo de acesso é refletida nos serviços; não se trata de uma aplicação que comunique exclusivamente por REST.

Os excertos seguintes são apresentados como texto com linhas numeradas. Os números correspondem às linhas dos ficheiros Java. O marcador [...] identifica linhas omitidas; a mudança visual de linha numa instrução longa não altera o código original.

4.2. Comunicação HTTP e consulta de dados

NetworkHandler.get recebe um endereço e cabeçalhos opcionais e devolve o corpo da resposta. A ligação usa GET, solicita JSON e estabelece tempos limite. A leitura do perfil e do histórico utiliza este método; na consulta de detalhes do Google são acrescentados cabeçalhos como a chave da API e a seleção dos campos pedidos.

Listagem 1 - NetworkHandler.java - excerto de public static String get

```
058 public static String get(String urlString,
Map<String, String> headers) throws Exception {
059 URL url = new URL(urlString);
060 HttpURLConnection conexao = (HttpURLConnection)
url.openConnection();
061
062 conexao.setRequestMethod("GET");
063                 conexao.setRequestProperty("Accept",
"application/json");
064 conexao.setConnectTimeout(10000);
065 conexao.setReadTimeout(10000);
// [...]
072
073 int responseCode = conexao.getResponseCode();
074 if (responseCode != HttpURLConnection.HTTP_OK) {
075 throw new Exception("Erro na ligação HTTP GET. Código:
" + responseCode);
// [...]
078 BufferedReader reader = new BufferedReader(new
InputStreamReader(conexao.getInputStream(),
StandardCharsets.UTF_8));
079 StringBuilder resposta = new StringBuilder();
080 String linha;
081
082 while ((linha = reader.readLine()) != null) {
083 resposta.append(linha);
084 }
085
086 reader.close();
087 conexao.disconnect();
088
089 return resposta.toString();
090 }
```

Nas linhas 59 a 66 são definidos o endereço, o método e os tempos limite. O bloco headers permite adaptar o pedido ao serviço usado. A verificação de responseCode rejeita respostas diferentes de HTTP 200. O BufferedReader lê o corpo em UTF-8, o ciclo while acrescenta as linhas recebidas e o método devolve a String resultante.

O método post utiliza um corpo JSON e é usado na pesquisa de ginásios e na consulta estruturada dos favoritos. Os serviços fornecem o cabeçalho Content-Type e os restantes parâmetros exigidos pelo destino. Os dados enviados são convertidos para bytes UTF-8 antes da escrita no fluxo da ligação.

Listagem 2 - NetworkHandler.java - excerto de public static String post

```
017 public static String post(String urlString, String
jsonBody, Map<String, String> headers)
throws Exception {
018 URL url = new URL(urlString);
019  HttpURLConnection conexao = (HttpURLConnection)
url.openConnection();
020
021 conexao.setRequestMethod("POST");
022 conexao.setConnectTimeout(10000);
023 conexao.setReadTimeout(10000);
024 conexao.setDoOutput(true);
// [...]
032 if (jsonBody != null) {
033 try (OutputStream os = conexao.getOutputStream()) {
034              byte[]              input              =
jsonBody.getBytes(StandardCharsets.UTF_8);
035 os.write(input, 0, input.length);
036 }
037 }
```

Os pedidos HTTP são chamados a partir de threads de trabalho nos serviços. Se ocorrer uma falha de rede ou um código de resposta não aceite, a exceção é tratada pelo serviço e comunicada ao ecrã através de um callback de erro.

4.3. Atualização e remoção de dados

As operações de escrita são centralizadas em NetworkHandler.patch e NetworkHandler.delete. O primeiro é utilizado para guardar favoritos e atualizar o histórico de visitas. O segundo remove o documento de um favorito. Não existe uma operação PUT no código analisado.

Listagem 3 - NetworkHandler.java - excerto de public static String patch

```
092 public static String patch(String urlString, String
jsonBody, Map<String, String> headers)
throws Exception {
093 URL url = new URL(urlString);
094  HttpURLConnection conexao = (HttpURLConnection)
url.openConnection();
095
096 conexao.setRequestMethod("POST");
097 conexao.setRequestProperty("X-HTTP-Method-Override",
"PATCH");
098 conexao.setConnectTimeout(10000);
099 conexao.setReadTimeout(10000);
100 conexao.setDoOutput(true);
101
// [...]
107
108 if (jsonBody != null) {
109 try (OutputStream os = conexao.getOutputStream()) {
110             byte[]             input             =
jsonBody.getBytes(StandardCharsets.UTF_8);
111 os.write(input, 0, input.length);
112 }
113 }
114
115 int responseCode = conexao.getResponseCode();
116 if (responseCode != HttpURLConnection.HTTP_OK) {
```

Nas linhas 96 e 97, a implementação envia POST com o cabeçalho X-HTTP-Method-Override: PATCH. O corpo JSON contém os campos a gravar. No histórico de visitas, o endereço inclui updateMask.fieldPaths para limitar a atualização ao histórico e preservar os restantes campos do perfil.

Listagem 4 - NetworkHandler.java - excerto de public static String delete

```
134 public static String delete(String urlString,
Map<String, String> headers) throws Exception {
135 URL url = new URL(urlString);
136 HttpURLConnection conexao = (HttpURLConnection)
url.openConnection();
137
138 conexao.setRequestMethod("DELETE");
139 conexao.setConnectTimeout(10000);
140 conexao.setReadTimeout(10000);
141
// [...]
148 int responseCode = conexao.getResponseCode();
149 if (responseCode != HttpURLConnection.HTTP_OK &&
responseCode !=
HttpURLConnection.HTTP_NO_CONTENT) {
150 throw new Exception("Erro na ligação HTTP DELETE.
Código: " + responseCode);
151 }
152
153 StringBuilder resposta = new StringBuilder();
```

A remoção recebe o endereço do documento e aceita HTTP 200 ou HTTP 204. Os serviços distinguem a conclusão da operação, comunicada por onSucesso, das falhas comunicadas por onErro. Os callbacks permitem que a interface apresente mensagens e atualize o estado da operação sem incorporar a lógica de HTTP no ecrã.

4.4. Interpretação de JSON e conversão de objetos

JsonHandler interpreta as respostas da Google Places API e do Firestore. Na pesquisa de ginásios, deSerializeJson2ListPlaceGymDTO percorre o array places e produz List<PlaceGymDTO>. São extraídos o identificador, nome, localização, morada e restantes campos disponíveis. Na consulta de detalhes, a resposta inclui também classificações, fotografias e avaliações públicas.

Listagem 5 - JsonHandler.java - excerto de public static List<PlaceGymDTO> deSerializeJson2ListPlaceGymDTO

```
018        public        static        List<PlaceGymDTO>
deSerializeJson2ListPlaceGymDTO(String resp) throws
JSONException {
019 List<PlaceGymDTO> list = new ArrayList<>();
020 JSONObject jsonObject = new JSONObject(resp);
021
022 if (!jsonObject.has("places")) return list;
023
024          JSONArray          jsonResponse          =
jsonObject.getJSONArray("places");
025
026 for(int i = 0; i < jsonResponse.length(); i++){
027          JSONObject          jsonChildNode          =
jsonResponse.getJSONObject(i);
028
029 String id = jsonChildNode.optString("id", "");
030 String name = jsonChildNode.has("displayName") ?
jsonChildNode.getJSONObject("displayName").optString("te
xt", "Sem Nome") : "Sem Nome";
031
032 double lat = 0.0, lng = 0.0;
033 if (jsonChildNode.has("location")) {
034                          lat                          =
jsonChildNode.getJSONObject("location").optDouble("latit
ude", 0.0);
035                          lng                          =
jsonChildNode.getJSONObject("location").optDouble("longi
tude", 0.0);
036 }
037
038  double  rating  =  jsonChildNode.optDouble("rating",
0.0);
```

```
039             String              morada              =
jsonChildNode.optString("formattedAddress", "");
```

A partir da linha 18, o texto é convertido num JSONObject. O método procura o array places e percorre os seus elementos. Os objetos DTO preservam os dados recebidos sem depender dos componentes visuais. A leitura de perfis do Firestore segue outra estrutura: fields contém valores tipados, como stringValue e arrayValue.

Listagem 6 - Mapper.java - excerto de public static Ginasio toModel

```
014 public static Ginasio toModel(PlaceGymDTO dto) {
015 if (dto == null) return null;
016      if      (dto.getId()      ==      null      ||
dto.getId().trim().isEmpty()) {
017  throw  new  DataConsistencyException("Inconsistência
nos dados do ginásio: ID em
falta.");
018 }
019
020 Ginasio ginasio = new Ginasio(
021 dto.getId(),
022 dto.getNome(),
023 dto.getLatitude(),
024 dto.getLongitude(),
025 dto.getRating(),
026 dto.isAbertoAgora(),
027 dto.getMorada(),
028 dto.getNumeroAvaliacoes(),
029 dto.getReferenciaFoto()
030 );
```

Mapper.toModel valida a presença do identificador e cria o objeto Ginasio. As fotografias e avaliações são transferidas para o modelo quando existem. toModelList aplica a conversão à lista e filtra alguns locais que não correspondem ao tipo de estabelecimento pretendido. A mesma organização é usada para Perfil, Favorito e Visita, evitando que os ecrãs interpretem diretamente o JSON.

4.5. Localização threads e carregamento

MainActivity solicita as permissões de localização e utiliza FusedLocationProviderClient. Tenta obter a última localização conhecida e, quando esta não existe, pede uma localização atual. As coordenadas são enviadas a PlacesService.pesquisarGinasiosProximos, que pesquisa num raio de 10 000 metros e limita a resposta a 20 resultados ordenados pela distância.

Listagem 7 - PlacesService.java - excerto de public void pesquisarGinasiosProximos

```
028 public void pesquisarGinasiosProximos(LatLng posicao,
OnGymsLoadedListener listener) {
029 new Thread(() -> {
030 try {
031 Map<String, String> headers = new HashMap<>();
032 headers.put("Content-Type", "application/json");
033                         headers.put("X-Goog-Api-Key",
BuildConfig.GOOGLE_PLACES_API_KEY);
034 headers.put("X-Goog-FieldMask",
035
"places.id,places.displayName,places.location,places.for
mattedAddress,place
s.regularOpeningHours.openNow");
036
037 String jsonInputString = "{"
// [...]
051
052  String  respostaJson  =  NetworkHandler.post(URL,
jsonInputString, headers);
053 listener.onProgress(70);
054          List<PlaceGymDTO>          dtos          =
JsonHandler.deSerializeJson2ListPlaceGymDTO(respostaJson
);
055 List<Ginasio> ginasios = Mapper.toModelList(dtos);
056 listener.onProgress(85);
057
058 listener.onSuccess(ginasios);
059
060 } catch (Exception e) {
061 android.util.Log.e("PLACES_ERRO", "Erro no serviço: "
+ e.getMessage());
062 listener.onError(e);
063 }
```

```
064 }, "BetterGym-Places").start();
065 }
```

O método cria uma nova Thread na linha 29. Depois do pedido POST, onProgress(70) indica que a resposta foi recebida; após o tratamento dos DTO e a conversão dos modelos, onProgress(85) assinala a preparação dos resultados. O callback onSuccess entrega List<Ginasio> à Activity.

MainActivity utiliza runOnUiThread antes de alterar os componentes visuais. A operação desenharMarcadores acrescenta os ginásios ao mapa e associa cada marcador ao respetivo objeto. A barra passa por etapas de preparação do mapa, localização, pesquisa, receção e apresentação dos resultados. O valor de 100% indica o fim desse percurso, não a percentagem de bytes transferidos.

Quando as permissões são recusadas, a localização está indisponível ou o carregamento excede o tempo limite, são apresentadas mensagens e opções de recuperação. Um identificador de tentativa evita que uma resposta antiga substitua o estado de uma pesquisa posterior. As operações de rede dos serviços são executadas fora da thread principal; as operações dos SDK do Firebase utilizam as suas tarefas assíncronas.

4.6. Autenticação e gestão da conta

AuthService concentra o registo, o início de sessão e a alteração da palavra-passe. No registo, valida os campos obrigatórios, o formato do email, o comprimento do telefone e a confirmação da palavra-passe. A criação do utilizador no Firebase Authentication é seguida da gravação do perfil no Firestore. O resultado é comunicado à Activity apenas através dos callbacks definidos no serviço.

Listagem 8 - AuthService.java - excerto de public void fazerLogin

```
169 public void fazerLogin(String email, String password,
AuthCallback callback) {
170 if (email.isEmpty() || password.isEmpty()) {
171 callback.onErro(new InvalidDataException("Preenche o
email e a password!"));
172 return;
173 }
174
175 mAuth.signInWithEmailAndPassword(email, password)
176 .addOnCompleteListener(task -> {
177 if (task.isSuccessful()) {
178 FirebaseUser user = mAuth.getCurrentUser();
179 if (user != null) {
180 callback.onSucesso(user.getUid());
181 } else {
182 callback.onErro(new DataConsistencyException("Erro de
consistência:
utilizador sem sessão válida."));
183 }
184 } else {
185 callback.onErro(new InvalidDataException("Email ou
palavra-passe
incorretos!"));
186 }
187 });
188 }
```

fazerLogin rejeita campos vazios e envia as credenciais a signInWithEmailAndPassword. Quando a tarefa termina com sucesso, confirma que existe um FirebaseUser e devolve o UID. LoginActivity guarda esse identificador nas preferências de sessão, apresenta a confirmação e abre MainActivity. A palavra-passe não é guardada nas preferências da aplicação.

Na alteração da palavra-passe, PasswordValidator verifica os campos, a dimensão mínima, a confirmação e a diferença face à palavra-passe atual. AuthService executa reauthenticate antes de updatePassword e confirma que a sessão se mantém. As falhas são apresentadas no pop-up. Ao terminar sessão, PerfilService chama signOut; o fragmento limpa as preferências e regressa ao login.

O perfil pode incluir uma fotografia opcional. A imagem escolhida é comprimida por ImageHelper, convertida para Base64 e guardada no campo foto_url do documento de perfil. A consulta apresenta os dados pessoais e as contagens de visitas e favoritos; a fotografia pode ser alterada pelo utilizador e, na sua ausência, são mostradas as iniciais. Esta funcionalidade integra a gestão do perfil, sem alterar o mecanismo geral de autenticação.

4.7. Gestão de favoritos

FavoritosService utiliza o UID da sessão para identificar os favoritos do utilizador. Cada documento combina o UID com o identificador do ginásio. guardar regista essa associação, a data e o nome do estabelecimento. remover elimina o documento. verificarEstado permite apresentar a estrela preenchida ou vazia na ficha do ginásio.

Listagem 9 - FavoritosService.java - excerto de public void guardar

```
120 public void guardar(Ginasio ginasio, EscritaCallback
callback) {
121 new Thread(() -> {
122 try {
123 if (ginasio == null || ginasio.getId() == null ||
ginasio.getId().trim().isEmpty())
{
124 throw new InvalidDataException("Ginásio inválido para
guardar.");
125 }
126 String uid = uidAtual();
127 String docId = uid + "_" + ginasio.getId();
128 String url = BuildConfig.BASE_URL + "favoritos/" +
docId;
129
130 String dataAtual = DateHelper.obterDataAtualIso();
131
132 Map<String, String> headers = new HashMap<>();
133 headers.put("Content-Type", "application/json");
```

```
// [...]
144 NetworkHandler.patch(url, jsonBody, headers);
145
146 callback.onSucesso();
147 } catch (Exception e) {
148 callback.onErro(e);
149 }
150 }).start();
151 }
```

O serviço valida o ginásio e obtém a sessão antes de construir o endereço do favorito. A data é produzida por DateHelper.obterDataAtualIso. O PATCH é executado numa nova Thread; apenas depois de concluir é emitido onSucesso. Em caso de erro, a interface conserva o estado anterior e apresenta a mensagem correspondente.

A consulta da lista usa POST para executar uma structuredQuery com filtro user_id. JsonHandler produz List<FavoritoDTO>, o Mapper converte para List<Favorito> e os resultados são ordenados pela data de adição. FavoritosAdapter recebe a nova lista e chama notifyDataSetChanged. Durante uma remoção, definirOcupado impede que o mesmo item seja acionado repetidamente. Após sucesso, a lista é carregada de novo.

O corpo JSON de guardar é atualmente construído por concatenação de texto. Esse comportamento deve ser considerado ao validar nomes de ginásios que contenham caracteres especiais. A construção de JSON com JSONObject, já utilizada no histórico, permite tratar esses valores sem os inserir manualmente na estrutura textual.

4.8. Registo e consulta de visitas

O registo de uma visita utiliza a câmara e a leitura de um QR Code. DetalheGinasioActivity verifica a permissão, abre o leitor e entrega o conteúdo a VisitasService.validarQrCodeGinasio. O serviço verifica o ginásio e o código antes de iniciar a gravação. Códigos rejeitados originam uma mensagem e não seguem para o pedido de escrita.

Listagem 10 - VisitasService.java - excerto de public void registarVisita

```
099 public void registarVisita(Ginasio ginasio,
EscritaCallback callback) {
100 new Thread(() -> {
101 try {
102 validarRegrasVisita(ginasio);
103
104 String uid = uidAtual();
105
106 String urlGet = BuildConfig.BASE_URL + "perfis/" +
uid;
107 String respostaJson = NetworkHandler.get(urlGet,
null);
108
109 PerfilDTO perfilDTO =
JsonHandler.deSerializeFirebaseJson2PerfilDTO(respostaJs
on);
110 List<VisitaDTO> historico = new ArrayList<>();
111 if (perfilDTO != null &&
perfilDTO.getHistorico_ginasios_visitados() != null) {
112
historico.addAll(perfilDTO.getHistorico_ginasios_visitad
os());
113 }
114
115 String dataAtual = DateHelper.obterDataAtualIso();
116
117 String nome = (ginasio.getNome() != null &&
!ginasio.getNome().trim().isEmpty())
118 ? ginasio.getNome().trim()
// [...]
122
```

```
123             String             jsonBody             =
JsonHandler.serializeHistoricoVisitasFirebaseJson(histor
ico);
124
125 String urlPatch = BuildConfig.BASE_URL + "perfis/" +
uid
126                                                      +
"?updateMask.fieldPaths=historico_ginasios_visitados";
127 Map<String, String> headers = new HashMap<>();
128 headers.put("Content-Type", "application/json");
129
130 NetworkHandler.patch(urlPatch, jsonBody, headers);
131
132 callback.onSucesso();
133 } catch (Exception e) {
134 callback.onErro(e);
135 }
```

O método obtém o perfil atual por GET e interpreta o histórico. Cria uma Visita com o identificador do ginásio, a data e o nome e acrescenta-a à lista. JsonHandler.serializeHistoricoVisitasFirebaseJson transforma essa lista no corpo do PATCH. A máscara de atualização restringe a escrita a historico_ginasios_visitados, mantendo os restantes dados do perfil.

A serialização utiliza JSONObject e JSONArray e representa os campos de acordo com a estrutura tipada do Firestore. A operação inversa, utilizada por obterVisitas, interpreta o histórico como List<VisitaDTO> e aplica Mapper.toModel a cada elemento. A lista é ordenada por data, da visita mais recente para a mais antiga, antes de ser entregue a HistoricoVisitasActivity.

A validação QR atual aceita o identificador, um texto que o contenha e o nome do ginásio. Também aceita um fragmento do identificador, uma regra que deve ser reforçada. A gravação do histórico usa leitura seguida de escrita da lista completa, pelo que duas atualizações simultâneas exigem validação adicional.

4.9. Serialização e apresentação dos resultados

A construção de JSON é necessária nas escritas que atualizam os dados do utilizador. O exemplo seguinte apresenta parte de serializeHistoricoVisitasFirebaseJson, que recebe List<VisitaDTO> e organiza os valores num documento Firestore. Ao contrário da desserialização, este método produz o texto que será enviado ao servidor.

Listagem 11 - JsonHandler.java - excerto de public static String serializeHistoricoVisitasFirebaseJson

```
183 public static String
serializeHistoricoVisitasFirebaseJson(List<VisitaDTO>
visitas) throws
JSONException {
184 JSONObject root = new JSONObject();
185 JSONObject fields = new JSONObject();
186 JSONObject histObj = new JSONObject();
187 JSONObject arrayValue = new JSONObject();
188 JSONArray values = new JSONArray();
189
190 if (visitas != null) {
191 for (VisitaDTO v : visitas) {
192 JSONObject item = new JSONObject();
// [...]
195
196 JSONObject gIdObj = new JSONObject();
197 gIdObj.put("stringValue", v.getGinasio_id() != null ?
v.getGinasio_id() : "");
198 itemFields.put("ginasio_id", gIdObj);
199
200 JSONObject dVisitaObj = new JSONObject();
201 dVisitaObj.put("stringValue", v.getData_visita() !=
null ? v.getData_visita() :
"");
202 itemFields.put("data_visita", dVisitaObj);
203
204 if (v.getNome_ginasio() != null &&
!v.getNome_ginasio().isEmpty()) {
205 JSONObject nObj = new JSONObject();
// [...]
214 }
215
```

```
216 arrayValue.put("values", values);
217 histObj.put("arrayValue", arrayValue);
218 fields.put("historico_ginasios_visitados", histObj);
219 root.put("fields", fields);
220
221 return root.toString();
222 }
```

Na linha 183 é recebida a lista de visitas. Os objetos root, fields e arrayValue representam a estrutura do documento; o ciclo cria um mapValue por visita. Os identificadores e as datas são escritos como stringValue. No fim, root.toString() produz o corpo do pedido.

A apresentação das coleções é feita por adapters. FavoritosAdapter e VisitasAdapter atualizam listas e notificam a interface. FotosAdapter utiliza um ViewPager2 para percorrer as referências de fotografias; cada imagem é obtida numa thread de trabalho e aplicada à ImageView na thread principal. A referência associada à imagem permite evitar que uma resposta atrasada seja colocada numa posição reutilizada.

As avaliações públicas são recebidas com os detalhes do Google e convertidas para Avaliacao. AvaliacaoAdapter apresenta o autor, a nota e o comentário. Não existem operações de escrita de avaliações na BetterGym. Os ecrãs distinguem carregamento, resultados, listas vazias e erros, permitindo repetir as consultas quando a operação não é concluída.

5. Testes

A validação da BetterGym abrange testes unitários e testes funcionais de ponta a ponta. Os testes unitários verificam métodos e regras isoladas. Os testes funcionais verificam os percursos dos casos de uso através da interface e a sua integração com os serviços utilizados pela aplicação.

5.1. Testes unitários

Foram executados os 35 testes unitários existentes no projeto, através de testDebugUnitTest. Todos terminaram com sucesso, sem falhas ou erros. A compilação debug também foi concluída com sucesso. As tabelas seguintes descrevem todos os métodos de teste executados e o respetivo resultado.

5.1.1. ExceptionsTest

Teste	Condição verificada e resultado esperado	Resultado Obtido
testDataConsistencyException_MetodoWhat	Verifica a mensagem, what() e a herança de RuntimeException para DataConsistencyException.	Aprovado
testDuplicatedDataException_MetodoWhat	Verifica a mensagem, what() e a herança de RuntimeException para DuplicatedDataException.	Aprovado
testInvalidDataException_MetodoWhat	Verifica a mensagem, what() e a herança de RuntimeException para InvalidDataException.	Aprovado

5.1.2. HelpersTest

Teste	Condição verificada e resultado esperado	Resultado Obtido
testPasswordValidator_MenosDeSeisCaracteres_RetornaErro	Uma nova palavra-passe com cinco caracteres deve devolver erro.	Aprovado
testDateHelper_ObterDataAtualIso_FormatoValido	O texto produzido deve conter T e terminar em Z; não valida todos os componentes do formato.	Aprovado
testPasswordValidator_Sucesso_RetornaNull	Dados válidos e confirmação coincidente devem devolver null, sem erro de validação.	Aprovado
testUtils_ValorOuTraco_ProtecaoContraNulos	Verifica o valor normal e a substituição de valores nulos ou em branco por um traço.	Aprovado
testDateHelper_FormatarVisita_ComDataValidaEInvalida	Verifica o prefixo Visitado a e o texto Data desconhecida quando a data é nula.	Aprovado

testPasswordValidator_NovaIgualAtual_RetornaErro	A nova palavra-passe igual à atual deve devolver erro.	Aprovado
testPasswordValidator_ConfirmacaoDiferente_RetornaErro	A confirmação diferente da nova palavra-passe deve devolver erro.	Aprovado
testDateHelper_ConverterIsoParaDate_InvalidoRetornaNull	Datas nulas, vazias ou ilegíveis devem produzir null.	Aprovado
testUtils_Iniciais_CasosCompletos	Verifica iniciais para nomes com uma ou várias palavras e para valores nulos ou vazios.	Aprovado
testDateHelper_ConverterIsoParaDate_ComEMilissegundos	Converte datas ISO com e sem milissegundos para Date.	Aprovado
testUtils_FormatarRating	Verifica a classificação e o número de avaliações, ou a mensagem de ausência de avaliações públicas.	Aprovado
testPasswordValidator_CamposVazios_RetornaErro	Campos nulos ou vazios devem devolver a mensagem de preenchimento obrigatório.	Aprovado

5.1.3. MapperTest

Teste	Condição verificada e resultado esperado	Resultado Obtido
testToModelPlaceGymDTO_NuloRetornaNulo	Um PlaceGymDTO nulo deve originar um modelo nulo.	Aprovado
testToModelList_FiltragemInteligenteDeLocais	Mantém três ginásios e exclui os exemplos de estádio e piscina.	Aprovado
testToDTOPlaceGym_MapeamentoCompleto	Verifica os campos efetivamente afirmados na conversão de Ginasio para PlaceGymDTO.	Aprovado
testToModelVisitaDTO_SemGinasioId_LancaDataConsistencyException	Uma visita sem identificador de ginásio deve lançar DataConsistencyException.	Aprovado
testToModelPlaceGymDTO_MapeamentoCompleto	Verifica os dados do ginásio, coordenadas, classificação, estado de abertura e fotografias na conversão DTO para Model.	Aprovado
testToModelPerfilDTO_IdNulo_LancaDataConsistencyException	Um identificador de perfil nulo deve lançar DataConsistencyException.	Aprovado

testToModelVisitaDTO_MapeamentoCompleto	Verifica identificador do ginásio, data e nome na conversão de VisitaDTO.	Aprovado
testToModelPlaceGymDTO_IdEmFalta_LancaDataConsistencyException	Um DTO de ginásio sem identificador deve lançar DataConsistencyException.	Aprovado
testToModelPerfilDTO_MapeamentoComHistorico	Verifica o identificador, nome, username e a conversão de duas visitas no perfil.	Aprovado

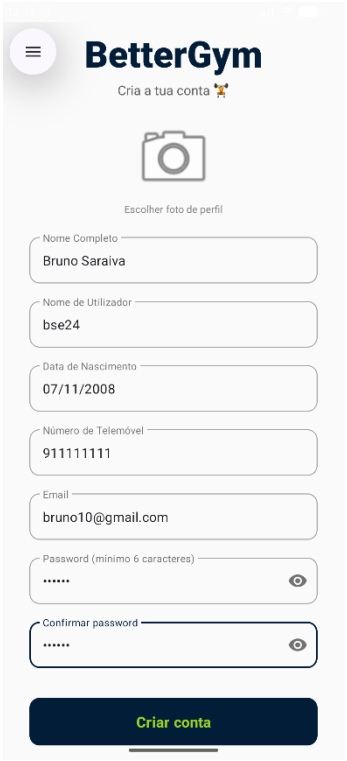
5.1.4. VisitasServiceTest

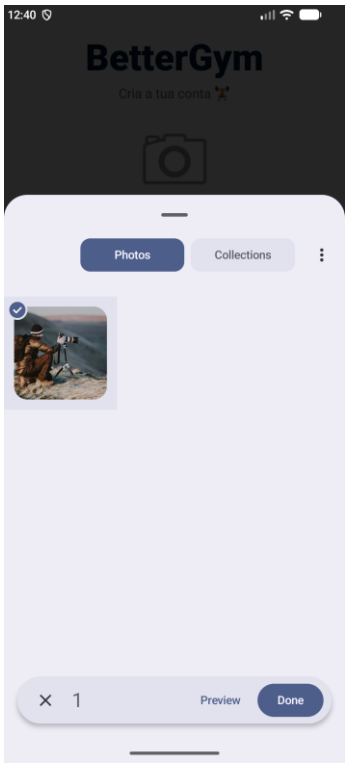
Teste	Condição verificada e resultado esperado	Resultado Obtido
testValidarQrCode_QrCodeDeOutroGinasio_LancaExcecao	O exemplo de QR de outro ginásio deve ser rejeitado.	Aprovado
testValidarQrCode_QrCodeComPrefixoCustomizado_Sucesso	Um QR com BETTERGYM_CHECKIN e o identificador completo deve ser aceite.	Aprovado
testValidarRegrasGinasio_GinasioFechado_LancaExcecao	Um ginásio fechado deve ser rejeitado com mensagem correspondente.	Aprovado
testValidarQrCode_GinasioFechado_LancaExcecaoMesmoComQrCorreto	Mesmo com o ID correto, um ginásio fechado deve ser rejeitado.	Aprovado
testValidarQrCode_QrCodeComNomeDoGinasio_Sucesso	Um QR que contém o nome do ginásio deve ser aceite pela regra atual.	Aprovado
testValidarQrCode_QrCodeIdExato_Sucesso	Um QR igual ao identificador do ginásio deve ser aceite.	Aprovado
testValidarRegrasGinasio_GinasioAberto_Sucesso	Um ginásio com identificador e aberto não deve lançar exceção.	Aprovado
testValidarQrCode_QrCodeVazioOuNulo_LancaExcecao	O teste usa uma String vazia e espera InvalidDataException; não exercita null apesar do nome.	Aprovado
testValidarRegrasGinasio_GinasioIdVazio_LancaExcecao	Um ginásio com identificador em branco deve ser rejeitado.	Aprovado
testValidarRegrasGinasio_GinasioNulo_LancaExcecao	Um ginásio nulo deve ser rejeitado.	Aprovado

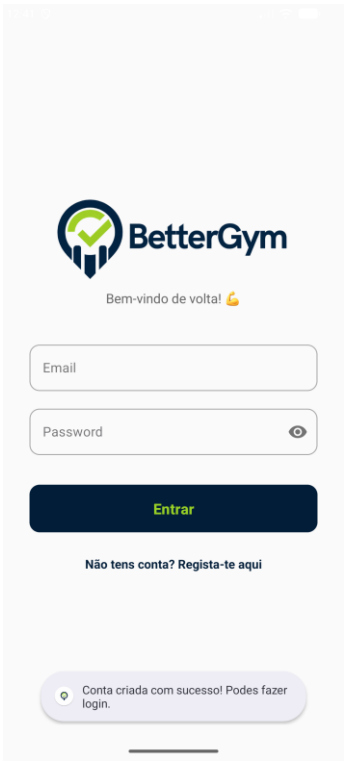
5.2. Testes funcionais

5.2.1. UC01 – Registar Conta

Procedimento: Preencher os dados, escolher uma fotografia e selecionar Criar conta.



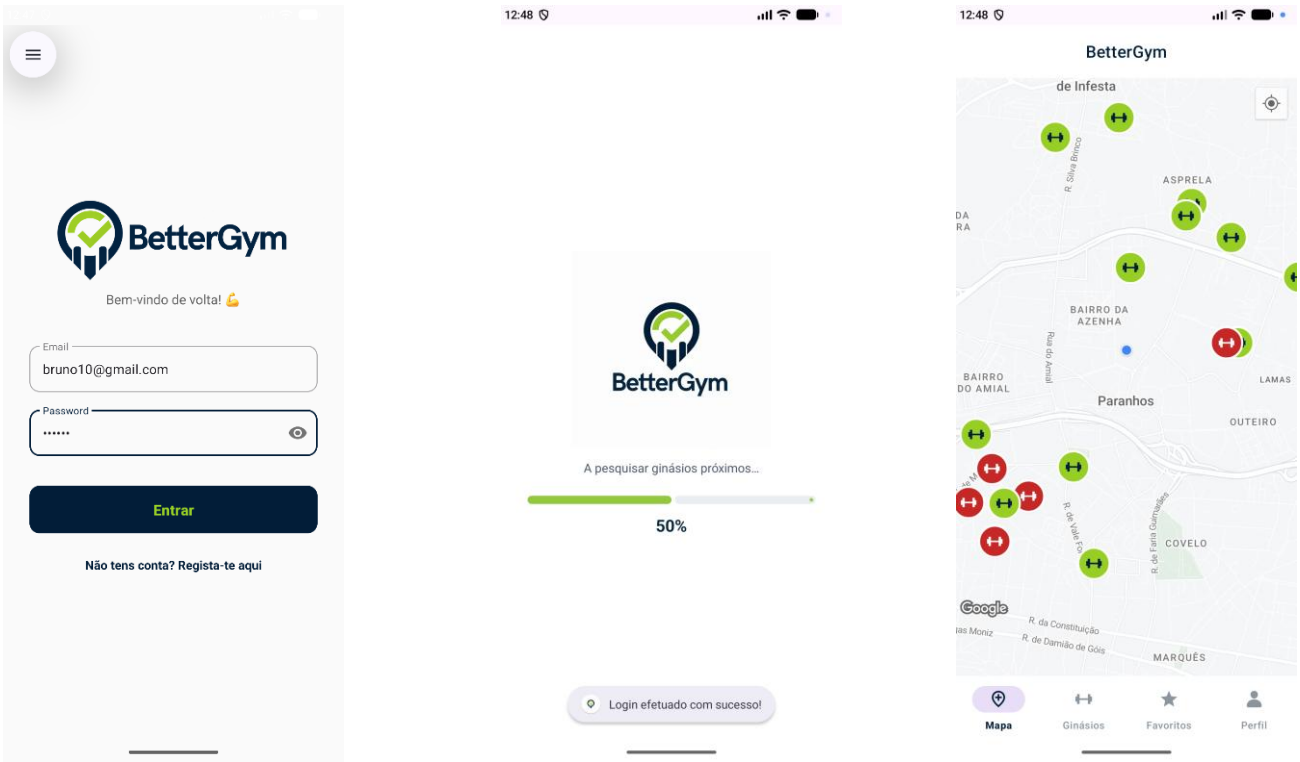




1. Formulário preenchido → 2. Fotografia selecionada → 3. Perfil após novo login

5.2.2. UC02 – Iniciar sessão

Procedimento: Introduzir email e palavra-passe e selecionar Entrar.



1. Dados de acesso → 2. Tela de carregamento → 3. Aplicação carregada

6. Conclusão

O desenvolvimento da BetterGym permitiu aplicar os conhecimentos adquiridos na unidade curricular de Desenvolvimento de Software e Sistemas Móveis na construção de uma aplicação Android em Java. Foram implementadas funcionalidades de autenticação, gestão do perfil e da fotografia, pesquisa de ginásios próximos, consulta de detalhes, fotografias e avaliações públicas do Google, gestão de favoritos e registo e consulta de visitas através de QR Code.

A compilação da aplicação e os 35 testes unitários executados terminaram com sucesso, confirmando os comportamentos abrangidos por esses testes. A validação completa dos percursos de utilização depende ainda da execução dos testes funcionais num dispositivo Android. Permanecem também aspetos a melhorar, nomeadamente o processamento das imagens fora da thread principal, a validação mais rigorosa dos códigos QR e o tratamento de atualizações simultâneas do histórico.

Os principais desafios técnicos estiveram relacionados com a integração dos serviços Firebase e Google, a comunicação REST, a interpretação de JSON e a conversão entre DTO e modelos. A utilização da localização e da câmara, o tratamento de permissões e a coordenação entre operações assíncronas e atualizações da interface exigiram igualmente atenção ao longo do desenvolvimento.

O projeto permitiu consolidar competências de programação orientada a objetos, desenvolvimento de interfaces Android e integração com serviços externos, estabelecendo uma base funcional que poderá ser aperfeiçoada através da correção das limitações identificadas e da validação em condições reais de utilização.